A Multi-Objective Genetic Programming Algorithm with α dominance and Archive for Uncertain Capacitated Arc Routing Problem

Shaolin Wang, *Student Member, IEEE,* Yi Mei, *Senior Member, IEEE,* and Mengjie Zhang, *Fellow, IEEE*

***Abstract*—The uncertain capacitated arc routing problem is an important combinatorial optimization problem with many applications in the real world. Genetic programming hyper heuristic has been successfully used to automatically evolve routing policies, which can make real-time routing decisions for uncertain capacitated arc routing problems. It is desired to evolve routing policies that are both effective and small/simple to be easily understood. The effectiveness and size are two potentially conflicting objectives. A further challenge is the objective selection bias issue, i.e., it is much more likely to obtain small but ineffective routing policies than the effective ones that are typically large. In this paper, we propose a new multi-objective genetic programming algorithm to evolve effective and small routing policies. The new algorithm employs the α dominance strategy with a newly proposed α adaptation scheme to address the objective selection bias issue. In addition, it contains a new archive strategy to prevent the loss of promising individuals due to the rotation of training instances. The experimental results showed that the newly proposed algorithm can evolve significantly better routing policies than the current state-of-the-art algorithms for uncertain capacitated arc routing problem in terms of both effectiveness and size. We have also analysed the evolved routing policies to show better interpretability.**

***Index Terms*—Capacitated Arc Routing, Genetic Programming, Hyper-Heuristic, Multi-Objective.**

I. Introduction

UNCERTAIN Capacitated Arc Routing Problem (UCARP) [1] is an important variant of the well-known Capacitated Arc Routing Problem (CARP) [2], which is a classic combinatorial optimisation problem and has been proven to be NP-hard [3]. CARP is to find a least cost routing plan to serve a set of edges (called tasks) by a set of vehicles subject to certain constraints, such as the total demand of the tasks served by a vehicle cannot exceed its capacity. CARP assumes that the environment is static, where all the parameters (e.g. task demand, serving cost) are fixed and known in advance. On the contrary, UCARP considers the parameters as random variables. The *Route failure* is one of the main challenges in UCARP. Typically, if the task demand is known in advance, it is straightforward to see whether the remaining capacity of the vehicle is sufficient to serve the next task. However, in UCARP, the task demand is not exactly known in advance. As a result, the vehicle will not know whether the remaining capacity is insufficient until the remaining capacity is used up. The vehicle has to refill the capacity to serve the failed task. This leads to an high recourse cost.

Recently, Genetic Programming Hyper-heuristic (GPHH) has been successfully utilised to handle UCARP by evolving routing policies which can guide a vehicle to determine which task to serve next once the vehicle is ready [4]. Comparing with traditional solution optimisation approaches, GPHH has shown its advantages in effectively handling route failures in real time [5]–[7]. However, the GP-evolved routing policies are usually too large and complex. A main reason is that GP tends to continuously increase the size of its individuals during the evolutionary process, which is known as *bloat* [8], [9]. A number of strategies have been proposed to deal with bloat [10]–[14], among which multi-objective GP (MOGP) [12], [14] is one of the promising techniques to optimise effectiveness and program size simultaneously.

There are two major challenges for developing MOGP to evolve effective and small routing policies for UCARP. First, GP is much more likely to generate small (but ineffective) individuals than effective (and typically large) ones. Under the traditional dominance relation, small ineffective individuals are more likely to be selected as parents and survive into the next generation. This is known as the *objective selection bias* issue [15]. Second, the fitness evaluation for routing policies is stochastic due to the training sample rotation [5], [16], i.e., it is calculated on a different small subset of training instances in each generation. The *stochastic fitness evaluation* can cause potentially good routing policies to be discarded if they happen to perform poorly on a small subset of training samples. This phenomenon becomes worse in MOGP, since most potentially good routing policies have large program size.

Existing studies have shown that the α-dominance method [15] is effective in addressing the objective selection bias issue, and the archive strategy [17] can well address the loss of potentially good individuals due to stochastic fitness evaluation. Therefore, it is reasonable to combine these two strategies in MOGP to further boost its effectiveness. However,

S. Wang, Y. Mei and M. Zhang are with the School of Engineering and Computer Science, Victoria University of Wellington, PO Box 600, Wellington 6140, New Zealand (e-mail: shaolin.wang@ecs.ac.nz; yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

Manuscript received XXX; revised XXX and XXX; accepted XXX. This work was supported in part by the Marsden Fund of New Zealand Government under Contract VUW1509 and Contract VUW1614 . (*Corresponding author: Yi Mei.*)

Shaolin Wang, Yi Mei, and Mengjie Zhang are with the Evolutionary Computation Research Group, School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: shaolin.wang@ecs.vuw.ac.nz; yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

Colour versions of one or more of the figures in this article are available online at XXX.

Digital Object Identifier XXX

no existing work has combined them together. Therefore, this paper proposes a new MOGP method combining the α-dominance and archive strategy.

Furthermore, this paper aims to address the following limitations of the existing α-dominance and archive strategies in MOGP for evolving routing policies. First, the effectiveness of the α-dominance method highly depends on the α value, which is challenging to determine. Intuitively, α should be increased if the population is biased to small ineffective individuals, and decreased if most individuals in the population are too large. However, the α adaption is non-trivial due to the difficulty of estimating the accurate boundaries of the effectiveness and program size. Second, the existing archive update strategies are not effective enough. It can include duplicate individuals in the archive and reduce the diversity of the archive.

To address the issue of sensitivity to the α value, in the new algorithm we propose a new scheme to estimate the relative program size of the population, and a new α adaption strategy guided by the relative program size estimation. To address the archive diversity issue, we propose a new archive update strategy that avoids including phenotypic duplicate individuals in the archive.

The overall goal of this paper is to propose a novel MOGP algorithm that can evolve a set of non-dominated routing policies that have different tradeoffs between the effectiveness and the size so that end users can select a routing policy based on their preference. To achieve this goal, we have the following specific research objectives:

1) To develop a new MOGP algorithm which combines the α-dominance and archive strategy (named αMOGP-a).
2) To develop a new α adaptation scheme that adaptively adjust the balance between effectiveness and size during the evolutionary process.
3) To develop a new archive strategy for identifying potentially useful routing policies to update the archive.
4) To verify the effectiveness of αMOGP-a by comparing with the current state-of-the-art on a wide range of UCARP instances, and verify the effectiveness of the newly proposed α adaptation scheme and archive strategy.
5) To interpret the routing policies evolved by αMOGP-a, and understand the behaviors of the GP-evolved routing policies.

The rest of this paper is organised as follows. Section II introduces the background. Section III describes the proposed approach. Section IV gives the experimental studies. Section V conducts the further analysis. Finally, Section VI gives the conclusions and possible future directions.

II. Background

A. Uncertain Capacitated Arc Routing Problem

UCARP can be represented by a connected undirected graph $G(V, E)$, where V and E are the set of vertices and edges. There is a depot $v_0 \in V$. Each edge $e \in E$ has a positive random deadheading cost $\bar{\varsigma}(e)$. Each edge e that need to be served (also called *task* t) has a positive random demand $\bar{d}(t)$ and a positive serving cost $sc(t)$. A fleet of vehicles with a positive capacity Q are allocated at the depot v_0 to serve the *tasks* $T \subseteq E$. The problem is to serve all the tasks in $T \subseteq E$ by the vehicles with the minimum cost under the following constraints: the vehicles must start and end at the depot; the total demand served by each vehicle between consecutive visits of the depot cannot exceed the vehicle's capacity.

UCARP contains random task demands and deadheading costs, each of which can have different sample values. As a result, there can be an infinite number of samples for an UCARP instance. In each UCARP instance sample, the task demand $\bar{d}(t)$ and deadheading cost $\bar{\varsigma}(e)$ take realised values. The actual demand of a task is not known until the task is served. This will lead to two main failures in UCARP, i.e., *route failure* and *edge failure*. A route failure occurs when the actual demand of a task exceeds the vehicle's remaining capacity. In this case, the vehicle must go back to the depot to reload the capacity and then come back to complete the service. An edge failure occurs when an edge becomes inaccessible suddenly, and the deadheading cost of this edge becomes infinity. As a result, it is necessary to repair the solution by finding a shortest detour under the current situation using pathfinding algorithms (e.g., Dijkstra's algorithm [18]).

A solution to a UCARP instance is represented as $S = (S.X, S.Y)$. $S.X = \{S.X^{(1)}, \ldots, S.X^{(j)}\}$ is a set of vertex sequences, where j is the number of vertex sequences (routes). $S.X^{(k)} = (S.x_1^{(k)}, \ldots, S.x_{L_k}^{(k)})$ stands for the kth route, where L_k represents the number of vertices in the kth route. $S.Y = \{S.Y^{(1)}, \ldots, S.Y^{(j)}\}$ is a set of continuous vectors, and $S.Y^{(k)} = (S.y_1^{(k)}, \ldots, S.y_{L_k-1}^{(k)})$ ($S.y_i^{(k)} \in [0, 1]$) is the fraction of demand served along the route $S.X^{(k)}$. For example, if $S.y_4^{(2)} = 0.9$, then $(S.x_4^{(2)}, S.x_5^{(2)})$ is a task and 90% of its demand is served at position 4 of the route $S.X^{(2)}$. Under the above representation, the problem can be formulated as follows.

$$\min \ \mathrm{E}_{\xi \in \Xi}[C(S_\xi)], \tag{1}$$

$$s.t. \ \left(S_\xi.x_i^{(k)}, S_\xi.x_{i+1}^{(k)}\right) \in E, \tag{2}$$

$$\left(S_\xi.y_i^{(k)}\right) \in [0, 1], \tag{3}$$

$$\sum_{i=1}^{L_k-1} d_\xi\left(S_\xi.x_i^{(k)}, S_\xi.x_{i+1}^{(k)}\right) \times S_\xi.y_i^{(k)} \leq Q, \forall k = 1, \ldots, j, \tag{4}$$

$$S_\xi.x_1^{(k)} = S_\xi.x_{L_k}^{(k)} = v_0, \quad \forall k = 1, 2, \ldots, j, \tag{5}$$

$$\sum_{k=1}^{j} \sum_{i=1}^{L_k-1} S_\xi.y_i^{(k)} \times S_\xi.z_i^{(k)}(e) = 1, \ \forall e : d_\xi(e) > 0, \tag{6}$$

$$\sum_{k=1}^{j} \sum_{i=1}^{L_k-1} S_\xi.y_i^{(k)} \times (1 - S_\xi.z_i^{(k)}(e)) = 0, \ \forall e : d_\xi(e) = 0, \tag{7}$$

where Eq. (1) is the objective function, which is to minimise the expected total cost $C(S_\xi)$ of the solution S_ξ over all samples $\xi \in \Xi$. The total cost of a solution on one sample

ξ is calculated by Eq. (8), where $\varsigma_\xi(u,v)$ is the realised deadheading cost between the node u to v in ξ. For any sample ξ, a feasible solution S_ξ is generated by a routing policy which gradually builds the solution on-the-fly. Eqs. (2) and (3) are the domain constraints of $S_\xi.X$ and $S_\xi.Y$. Eq. (4) is the capacity constraint, and d_ξ refers to the realised task demand in ξ. Eq. (5) indicates that all the routes start and end at the depot. Eq. (6) means that each task is served exactly once (the total demand fraction served by all vehicles is 1). $S_\xi.z_i^{(k)}(e)$ is equal to 1 if $\left(S_\xi.m_i^{(k)}, S_\xi.m_{i+1}^{(k)}\right) = e$, and 0 otherwise. $S_\xi.z_i^{(k)}(e)$ indicates whether the edge is a task or not. Eq. (7) ensures that any non-required edge is not served at all, i.e., its service fraction is zero everywhere in the solution.

$$C(S_\xi) = \sum_{k=1}^{j}\sum_{i=1}^{L_k-1}\left(\varsigma_\xi(S_\xi.x_i^{(k)}, S_\xi.x_{i+1}^{(k)})\right) + \sum_{t\in T}(sc(t) - \varsigma_\xi(t)) \tag{8}$$

B. α Non-dominated Sorting

In this paper, we use the α-dominance [19] which is a more general form of traditional dominance to identify the dominance relationship among individuals.

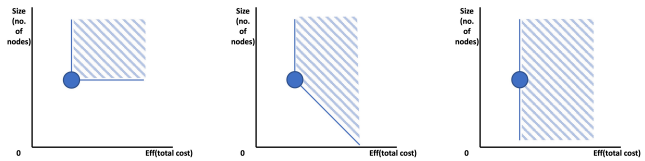
Specifically, given two solutions x and x', we say that x *α-dominates* x' if $\delta_{size}(x,x') \leq 0, \delta_{\mathit{eff}}(x,x') \leq 0$, and there are at least one inequality, where

$$\delta_{size}(x,x') = size(x) - size(x') + \alpha * (\mathit{eff}(x) - \mathit{eff}(x')) \tag{9}$$

$$\delta_{\mathit{eff}}(x,x') = \mathit{eff}(x) - \mathit{eff}(x') \tag{10}$$

where $size()$ refers to the size (number of nodes) in a solution and $\mathit{eff}()$ refers to the effectiveness of a solution.

Fig. 1 shows the how the dominance area changes when α value changes. It can be seen that when α equals 0, the dominance area is same as traditional dominance criteria in which two objectives are treated equally. When the α is increased, the effectiveness is subsequently given more importance. When α equals positive ∞, it only considers the single objective effectiveness and ignore the size.



(a) α = 0 (b) α = 1 (c) α = ∞

Fig. 1. The dominance area when α changes.

C. Related Work

1) Approaches to CARP: A variety of approaches have been proposed to solve CARP. The *Exact* approaches [2], [20] were firstly applied. The advantage of the exact approaches is that they can get the optimal solutions. However, they can only be applied to small-sized instances due to their high computational complexity. Constructive heuristics, such as path scanning heuristic [21], augment-merge heuristic [2], and Ulusoy's split heuristic [22], were proposed to obtain reasonably good solutions in a short time. Constructive heuristics are generally much faster than the exact approaches because the one-pass solution construction process typically has a linear complexity. In contrast, to guarantee optimality, the exact approaches have exponential complexity for NP-hard problems such as UCARP. Meta-heuristic approaches, such as tabu search [23], [24], genetic algorithms [25], memetic algorithms [26]–[28] and ant colony optimisation [29]–[31], were also applied to CARP. Meta-heuristic approaches usually iteratively improve one or more initial solutions. They can also provide promising solutions within a limited time budget. Since meta-heuristic algorithms can take the solution from constructive heuristics as their initial solution, they are not worse than constructive heuristics. Transfer learning techniques, such as meme learning and selection [32], memetic computing paradigm [33], are also utilised to enhance the problem solving capability for CARP. These approaches can capture useful structures or latent patterns from previous experiences of problem-solving, and transfer the structures and patterns to future evolutionary search. As a result, the effectiveness and search efficiency can be improved.

2) Approaches to UCARP: The CARP in reality can contain a variety of different uncertainties. Liu et al. [34] gave a comprehensive review on the uncertain factors in CARP, and the corresponding approaches to address them. Liu et al. [34] summarised four important uncertain factors in CARP, i.e., stochastic task demand, stochastic edge deadheading cost, stochastic task presence and stochastic edge existence, all of which are important to be considered. Thus, in this paper, we aim to solve the UCARP [1] model that considers all the above four uncertain factors.

There are two types of approaches to the UCARP model with the four uncertain factors. The first one aims to find solutions that are expected to handle all the possible realisations of the random variables, which can be namely "*proactive*" approaches. The second one uses Genetic Programming Hyper-Heuristics (GPHH) to evolve routing policies that construct solutions gradually in real time, which can be namely "*reactive*" approaches.

Typically, proactive approaches output a pre-planned solution and a recourse operator. They use an optimisation algorithm to generate the pre-planned solution, and then a recourse operator to repair the solution when failure, such as route failure, occurs. The performance of these approaches is highly dependent upon the accuracy of the prediction of the stochastic environment as well as the effectiveness of the recourse operators.

Fleury et al. [35] adapted the best method published for CARP, i.e.,hybrid genetic algorithm (HGA), to CARP with stochastic task demand (SCARP). HGA not only considers the solution cost but also the robustness of solutions. Wang et al. [36] proposed a new MA for UCARP. This new MA contains an integrated fitness function and a large step-size local search operator. It makes use of the integrated fitness

TABLE I
CLASSIFICATION OF APPROACHES TO (U)CARP.

Problem	Category	Literature
CARP	Exact approaches	[2], [20]
	Constructive heuristics	[2], [21], [22]
	Metaheuristics	[23]–[33]
UCARP	Proactive approaches	[35]–[38], [40], [41]
	Reactive approaches	[5], [7], [11], [42]–[49]

function to direct the search process while the large step-size local search operator prevents MA from trapping in the local optima. Babaee et al. [37] proposed hybrid meta-heuristic approach integrated with a mix integer programming model to deal with the demand uncertainty. The hybrid meta-heuristic approach utilises simulated annealing as the optimiser and a constructive heuristic to provide initial solution. Wang et al. [38] developed an Estimation of Distribution Algorithm with Stochastic Local Search (EDASLS) for UCARP. The EDASLS aims to get solutions that can perform well over a set of UCARP instances. It is developed based on the edge histogram based sampling algorithm [39]. A novel stochastic local search is proposed and integrated. The stochastic local search can effectively handle the uncertainties in UCARP. Tong et al. [40] proposed a generalised meta-heuristic framework for UCARP. In addition, Tong et al. [41] proposed a hybrid local search framework which can handle the small dynamic changes in UCARP.

In contrast, reactive approaches produce a routing policy without needing any pre-planned solution. GP is typically used to evolve a population of routing policies, and the best evolved routing policy can be applied to unseen/future scenarios to make routing decisions in real time.

Weise et al. [42] firstly adapted a GPHH to a single-vehicle UCARP and showed that routing policies evolved by GPHH could outperform manually designed routing policies. After that, to improve the effectiveness of GPHH, Liu et al. [5] proposed a novel pool filer to omit irrelevant candidate tasks during the decision-making process. In real world, tasks in UCARP are usually served by a fleet of vehicles. Thus, the model was extended from a single-vehicle to multi-vehicle version by Mei et al. [7]. MacLachlan et al. [43] improved the GPHH by proposing a novel pool filer which can prevent generating too small routes and an effective look-ahead terminal which can prevent route failure. Then, MacLachlan et al. [44] examined the advantage of vehicle collaboration in handling the uncertain environment. To speed up the training efficiency, Ardeh et al. [45], [46] applied transfer learning techniques to GPHH. Liu et al. [47] proposed a new paradigm for GPHH. The new paradigm utilises GP and cooperative coevolution to evolve a task sequence and a recourse policy simultaneously. Recently, Wang et al. [11], [48], [49] take the interpretability of the GP-evolved routing policies into consideration by evolving smaller routing policies. Table I shows the classification of the approaches to CARP and UCARP.

3) Size Reduction in GP: In general, a smaller GP tree tends to have better interpretability and generalisation [50]. GP trees are often evolved by randomly swapping the subtrees of the parent nodes in crossover and replacing the subtrees with randomly generated subtrees in mutation. Without careful control over the size of the trees, GP search often results in large and complex trees, which are difficult to interpret [49]. It is important to develop algorithms to reduce the size and complexity of the evolved program trees without compromising on effectiveness.

A number of methods have been proposed to control the size of the GP tree, the most straightforward of which is to add a limit to the size or depth of the tree [10]. The advantage of this method are that it is simple and does not require a complex implementation. However, it also has some limitations. Firstly, it is not easy to pre-define a reasonable limit on size or depth [14]. Secondly, limiting the size or depth can also limit the search capability of the GP. *Parsimony pressure* [12] is another commonly used method. It is implemented by adding a penalty term to the fitness function. The larger (size) the individual, the larger the penalty. In this way, smaller individuals get better fitness and are more likely to be selected as parents. Parsimony pressure does not affect the search ability of the GP as much as the size or depth limit, but it is also difficult to define a reasonable coefficient for the penalty term in advance. GP tree simplification is an alternative way to reduce the GP tree size. The basic idea of GP tree simplification is to remove the redundant part in the GP tree. As a result, it is possible to get smaller GP trees. Hooper et al. [51] firstly utilised the expression simplification, which contains a set of simplification rules to simplify the trees. Zhang and Wong [52] made improvements by using algebraic tree simplification with the hashing techniques. Their methods significantly speed up the simplification process. However, there are still some limitations to their approach, i.e., some redundant parts cannot be found and removed. To overcome this limitation, Kinzett et al. [13] and Song et al. [53] proposed numerical simplification methods. Kinzett et al. [13] removed a node or a subtree if its numerical contribution is smaller than a predefined threshold. Song et al. [53] replaced the parent node with its child node if the difference between their outputs is below a predefined threshold. In addition to these simplification methods, Wang et al. [49] proposed a novel niching GP simplification method. This method can detect redundant part of a GP tree and simplify the GP tree automatically by comparing GP trees in same niche.

All the above methods are single-objective methods which integrated some mechanism to control the GP tree size. In contrast, multi-objective algorithm can optimise effectiveness and tree size simultaneously without needing to set parameters. There have been a variety of Evolutionary Multi-Objective Optimisation (EMO) methods such as NSGA-II [54], SPEA2 [14] and MOEA/D [55]. Multi-Objective GP (MOGP) have been applied to solve a variety of problems, such as classification [56], feature extraction [57], dynamic job shop scheduling problems [58], [59], drug design [60], [61] and energy consumption forecast [62]. However, existing MOGP methods cannot effectively handle the objective selection bias issue and the issue of the loss of potentially good individuals in UCARP. Thus, it is necessary to develop new algorithms to handle these issues.

III. The Proposed Algorithm

A. Overall Framework

Fig. 2 shows the overall framework of the newly proposed αMOGP-a. Firstly, a population is randomly initialised and an empty archive is created. Each individual, which is represented as a GP tree, is evaluated based on a set of training instances (simulations). Specifically, the size (number of nodes) and the effectiveness (the mean total cost of the solutions generated for the training instances) are calculated for each individual. After that, an α non-dominated sorting is carried out to sort all the individuals in the population. The tournament selection is utilised to select parents to generate offspring. The offspring population is combined with the parent population. This follows the process of NSGA-II [54]. As we use the training sample rotation in UCARP, the parents in the combined population are re-evaluated on the instances used to evaluate the offspring. The boundary of individual size will be identified and updated. The boundary is utilised to identify the bias. The combined population is then sorted using the α non-dominated sorting and the non-dominated individuals are stored in the archive. The individuals in the archive will be involved in the breeding process. The utilisation of the archive will be discussed in Section III-D. The α parameter is then updated based on the distribution of the population. The α dominance will be discussed in Section II-B. The α adaptation scheme will be introduced in Section III-C. Then, the top N (N is the population size) individuals will be selected to next generation. When the stopping criteria are met, the archive individuals will be re-evaluated using a validation set of samples which is different from the training and test samples. A traditional non-dominated sorting is applied to the archive and the final non-dominated solutions are returned.

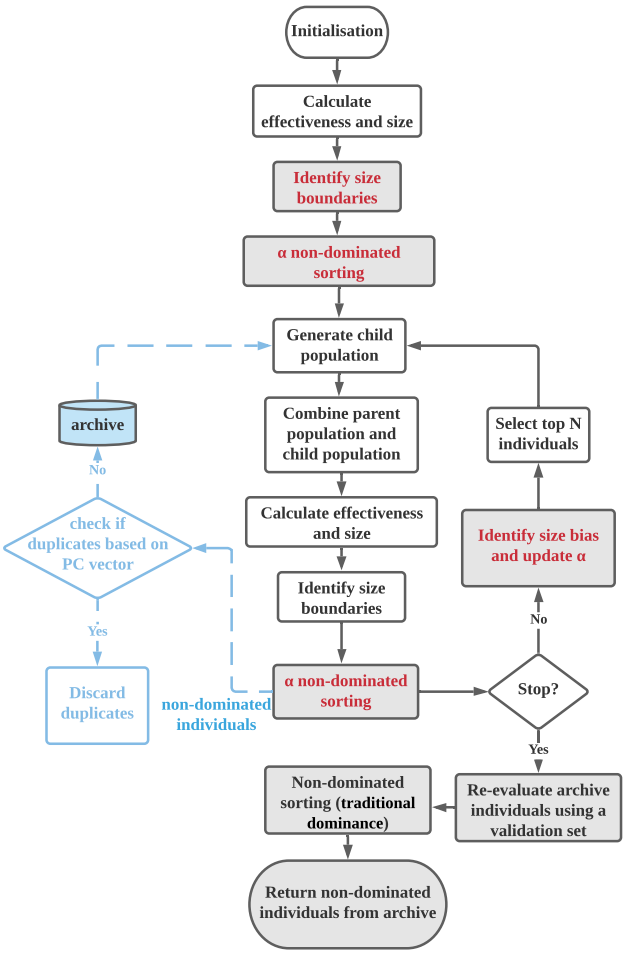


Fig. 2. The overall Framework of αMOGP-a.

B. Individual Representation and Evaluation

Typically when evolving routing policies using GPHH, each routing policy is represented as a GP tree. The GP tree is used as a priority function. We can use the priority function to calculate the priority value for each candidate task and then select the most prioritised task to serve next when a vehicle is idle. After all the candidate tasks are served, the routes are returned as the solution constructed by a routing policy. Each feature in the priority function is a state feature of the candidate task and vehicle. For example, a priority function "DEM + CFH" is shown in Fig. 3, where DEM refers to the demand of the candidate task and CFH refers to the cost from the current location to the candidate task. This priority function tends to select a candidate task that require less demand and close to the current location.

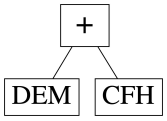


Fig. 3. An example of GP tree.

In this paper, we consider two objectives: the effectiveness and the size. Thus, given a routing policy RP, the effectiveness of RP refers to mean total cost of the solutions built by applying RP to the training instance samples, and the size of RP refers to the number of nodes in RP. Specifically,

$$\mathit{eff}(RP) = \frac{1}{|\mathcal{S}'|} \sum_{s \in \mathcal{S}'} tc(RP, s), \tag{11}$$

$$\mathit{size}(RP) = \text{number of nodes in } RP \tag{12}$$

where $\mathcal{S}'$ is a training set of instance samples, $|\mathcal{S}'|$ is the number of instance samples in S'. $tc(RP, s)$ stands for the total cost of the solution obtained by RP for the instance sample s. Algorithm 1 shows the evaluation process of αMOGP-a for a population at a generation.

C. α Adaptation Scheme

The α parameter controls the balance between the effectiveness and size during the search. To control the balance better, it is necessary to adaptively adjust the α value during the evolutionary process. In this paper, we propose a novel α adaptation scheme that can automatically adjust the α value based on the distribution of current α non-dominated solutions generated by α dominance. Here, we use size alone to estimate the bias, since it is difficult to estimate the range of effectiveness of routing policies during the search process

Algorithm 1: The individual evaluation in αMOGP-a.

Input: Training samples $\mathcal{S}$, An unevaluated population *pop*
Output: An evaluated population *pop*
Randomly sample a subset of samples $\mathcal{S}' \subseteq \mathcal{S}$;
// Evaluate *pop* using $\mathcal{S}'$
for *each routing policy* $RP \in pop$ **do**
  // Evaluate RP
  for *each instance sample* $s \in \mathcal{S}'$ **do**
    Create an empty solution;
    Put all candidate tasks in an empty queue;
    while *candidate tasks in queue is not empty* **do**
      if *vehicle is idle* **then**
        calculate priority value for each candidate task using RP;
        select the most prioritised task to serve;
        add the task to solution;
        update the queue;
      end
    end
    get the total cost of the generated solution, i.e., $tc(RP, s)$;
  end
  calculate $eff(RP)$ using Eq. (11);
  calculate $size(RP)$ using Eq. (12);
end
return evaluated population *pop*;

(e.g., a random routing policy can be very poor). If the current α non-dominated solutions is biased away from size, we will decrease the α value. Otherwise, if the current α non-dominated solutions is biased towards size, we will increase the α value.

The procedure of estimating the current bias to size is as follows. First, we calculate the mean size of the α non-dominated individuals in the current population, denoted as S_{pop}. Then, we compare S_{pop} with the historical range of the size $[S_{min}, S_{max}]$. If S_{pop} leans towards S_{min}, then we consider the current population to bias towards size. If S_{pop} is closer to S_{max}, then the current population tends to bias away from size. In addition, the amount of increase/decrease in α should depend on the degree of the bias. For example, we should increase α more if S_{pop} is closer to S_{min}.

Taking the above into account, we design the α adaptation scheme as follows.

$$\theta_{old} = \arctan(\alpha_{old}) \tag{13}$$

$$\theta_{new} = \theta_{old} + \frac{S_{max} + S_{min} - 2S_{pop}}{S_{max} - S_{min}} * \phi \tag{14}$$

$$\alpha_{new} = \tan(\theta_{new}) \tag{15}$$

where α_{old} and α_{new} are the α values before and after the update. θ_{old} and θ_{new} are angles corresponding to α_{old} and α_{new} (the angle between the horizontal line and the boundary of the dominance region induced by α in Fig. 3). ϕ is a step size which indicates how many degrees we need increase or decrease the angle to adjust the dominance area. In other words, the angle θ changes linearly from $-\phi$ to ϕ when S_{pop} changes from S_{min} to S_{max}. θ is unchanged if $S_{pop} = (S_{max} + S_{min})/2$, i.e., in the middle of the historical sizes.

Fig. 4 shows two examples of the α adaptation scheme. In the first example, the current α is 0, $S_{pop} = S_{min}$, and $\phi = 90°$. In this case, the current population is strongly biased towards size. Thus, we increase the angle by $90°$ (the maximum possible change) so that the subsequent search will ignore size and fully focus on effectiveness. In the second example, the current α is ∞ (angle is $90°$), $S_{pop} = S_{max}$, and $\phi = 45°$. In other words, the current population is biased away from size (the individuals are very large), and we need to decrease α. In this case, we decrease the angle by $45°$ (due to the step size ϕ), and α is decreased from ∞ to 1.

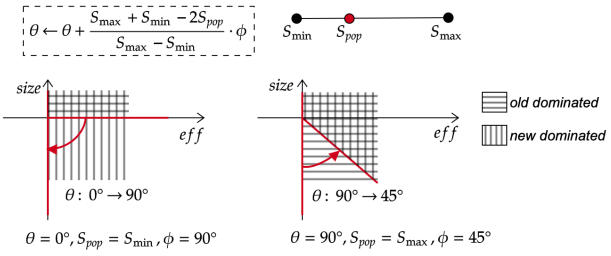


Fig. 4. The relationship between changes in the dominance area and the location the current Pareto front.

D. Archive Strategy

In this paper, the external archive is used to store the potentially good individuals occurred in the evolutionary process. At the end of each generation, the α non-dominated individuals are added to the archive if it is not a duplicate of any individual in the archive. The duplication checking process is based on the comparison of the Phenotypic Characterisation vector (PC vector) [63]. The phenotypic characterisation of an individual is an vector of decisions it makes in a list of representative decision situations. For each decision situation, the decision is the index of its selected task for the idle vehicle. If two individuals have similar PC vectors, they tend to have similar behaviour and effectiveness. Since the phenotypic characterisation is a vector of task indices, we use the following equation to calculate the distance between two PC vectors $\mathbf{p}_1$ and $\mathbf{p}_2$:

$$\Delta(\mathbf{p}_1, \mathbf{p}_2) = \sum_{i=1}^{L} \mathbb{I}_{p_{1i}=p_{2i}} \tag{16}$$

where $\mathbb{I}_{p_{1i}=p_{2i}} = 1$ if $p_{1i} = p_{2i}$, and $\mathbb{I}_{p_{1i}=p_{2i}} = 0$, otherwise. If the distance of two individuals is zero, we consider the two individuals to be duplicates. If a new individual and an existing individual in the archive are duplicates, then the new individual will replace the existing individual only if it has a smaller size. The archive updating process is shown in Algorithm 2.

To fully utilise the archive, we use it to provide parents for breeding. When the breeding process starts, the archive is merged with the current population to create a mating pool. Then, we select parents from the mating pool using tournament selection. Note that the individuals in the archive are evaluated on different instance samples from the current population, due to the instance rotation. To compare between the individuals in the archive and the current population, we assume that the individuals in the archive are slightly worse than the non-dominated individuals in the current population, but better than

other dominated individuals. If two selected individuals are both from current population, the better individual is the one that is dominated by fewer individuals or dominated by same number of individuals but has smaller crowding distance. If two individuals are both selected from the archive, we can randomly pick one as a parent. If one is from the archive and the other one is from the current population, as long as the individual from the current population is not a non-dominated individual, the archive individual will be chosen as a parent.

Typically, the final non-dominated solutions are chosen based on the performance of the solution on the training set at the last generation. However, this may lead to the final non-dominated solution being biased towards the training set of the last generation, which in turn results in the non-dominated solution performing poorly on the test set. In addition, to select non-dominated solutions from the archive, solutions in the archive should be re-evaluated. Therefore, at the end of the GP process, the archive will be passed to a validation set of samples to select the non-dominated solutions.

Algorithm 2: The updating process of the archive.

```
Input: The current population pop, the external archive, the
       population size popsize
Output: The updated archive
Obtain the α non-dominated individuals pop_nd in pop;
for ind ∈ pop_nd do
    Set duplicate = false;
    for ind' ∈ archive do
        calculate distance between ind and ind' using Eq.16;
        if distance = 0 then
            duplicate = true;
            if size(ind) < size(ind') then
                replace ind' with ind
            end
            break;
        end
    end
end
return archive;
```

E. Algorithm Complexity

The complexity of αMOGP-a is composed of (1) fitness evaluation (2) the α non-dominate sorting, (3) archive update. At each generation, The fitness evaluation includes applying each routing policy to K training instances to construct solutions. The solution construction has a complexity of $O(T^2S)$, T is the number of tasks and S is the program size. Hence, the complexity of the fitness evaluation is $O(NKT^2S)$, where N is the population size. After the fitness evaluation, the α non-dominate sorting has a complexity of $O(MN^2)$, where $M = 2$ is the number of objectives. The new α adaption scheme requires to calculate the min and max program size of the population. Its complexity is $O(N)$, thus can be ignored. The update of the archive requires to calculate the phenotypic vectors of the α non-dominate individuals and compare them with that of the archive. Its complexity is $O(N_{nd}LS + N_{nd}AL)$, where $N_{nd} \ll N$ is the number of α non-dominate individuals, $L \approx O(1)$ is the number of decisions in the phenotypic vector, and $A \approx O(N)$ is the number of individuals in the archive. We can see that $NKT^2S \gg N_{nd}LS + N_{nd}AL$. Thus, the complexity of αMOGP-a at each generation is $O(NKT^2S + MN^2 + N_{nd}LS + N_{nd}AL) = O(NKT^2S + MN^2)$, which is the same as that of the standard MOGP with non-dominated sorting.

IV. Experimental Studies

To verify the effectiveness of the newly proposed αMOGP-a, we design two sets of experiments:

1) Compare with the current state-of-the-art MOGP algorithm for UCARP (TSNSGPII-a [17]), to verify the quality of the obtained Pareto front, using Hyper Volume (HV) and Inverted Generational Distance (IGD).
2) Compare with the state-of-the-art algorithms for reducing program size (i.e., bloat control methods, such as Tarpeian, linear parametric parsimony pressure (LPPP), double tournament (DT) in the literature [64] and GPHH-N [49]), to verify the effectiveness and size of the obtained most effective routing policy.

A GP run contains two main phases, i.e., training and test. In the training phase, routing policies are trained on 350 training samples (250 samples for training and 100 sample for validating). The 250 samples are divided into 50 subset. Each subset contains 5 training samples. For each generation, one subset is used for fitness evaluation. At the end of training process, 100 validating samples are used to select final Pareto front from the archive. In the test phase, the final Pareto front is tested on 500 test samples. All the samples share the same graph topology of a UCARP instance but are generated by randomly sampling the stochastic task demands and deadheading costs.

A. Datasets and Parameter Setting

In this paper, αMOGP-a is verified on two commonly used UCARP datasets, i.e., the *Ugdb* and *Uval* [1], [65]. The *Ugdb* dataset mainly contains small instances that contains at most 55 tasks that require to be served by at most 5 vehicles. The *Uval* contains medium and large instances that have 34 to 97 tasks that require to be served by at most 10 vehicles. In addition, we also evaluate our algorithm on a real-world UCARP instance extracted from the map of the Hefei city in China, namely *Uhefei* [66]. The instance contains 850 vertices and 1212 edges. 121 edges are tasks which are served by 7 vehicles.

The function set is set to $\{+, -, \times, /, \min, \max\}$, where the "/" (protected division) operator returns 1 if divided by 0. The terminal set is shown in Table II. The initialisation method of the population for all compared algorithms is Ramped Half-and-Half. All these parameter settings are commonly used in the GPHH literature [7], [67], [68]. The population size for MOGP algorithms are set to 500. For single-objective GP algorithm, it is set to 1000. This is because MOGP combines parent and offspring population to generate a new combined population which will double the original population size. For fair comparison, the population size of the MOGP algorithms is set as the half of the single-objective algorithm. The maximal number of generations is set to 50. The tournament selection size is 2. The ratio of crossover and mutation operator

TABLE II
THE TERMINAL SET IN THE EXPERIMENTS.

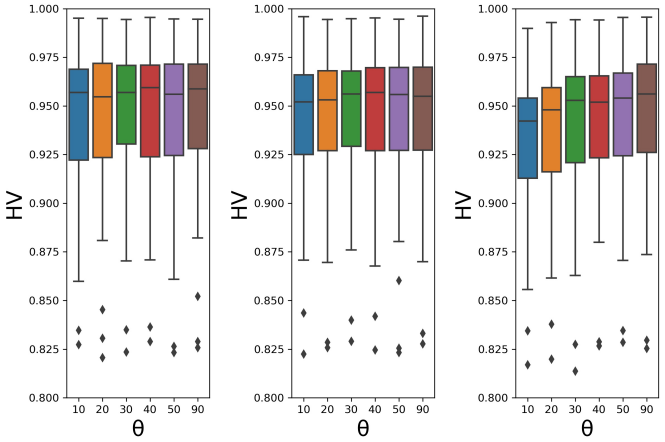
Terminal	Description
CFH	cost from the candidate task to the current location
CFD	cost from the head node of the task to the depot
CFR1	cost from the closest other route to the candidate task
CTT1	cost from the candidate to its closest remaining task
CTD	cost from the depot to the candidate task
CR	cost from the depot to the current location
DEM	expected demand of the candidate task
DEM1	demand of the closest unserved task to the candidate task
DC	deadheading cost of the candidate task
FULL	fullness (served demand over capacity) of the vehicle
FRT	fraction of unserved tasks
FUT	fraction of unassigned tasks
RQ	remaining capacity of the vehicle
RQ1	remaining capacity of the closest alternative route
SC	cost of serving the candidate task
ERC	a random constant value

is set as 0.85 : 0.15. The maximal tree depth of all GP algorithm is set to 8. The elitism size of the single-objective GP is set to 10. All the compared MOGP algorithms are non-dominated sorting based, thus there is no need to use elitism. The Evolutionary Computation Java (ECJ) package [69] is utilised to implement all the algorithms. The results are collected based on 30 independent runs for each algorithm on each UCARP instance.

B. Parameter Sensitive Analysis

For the α dominance, the initial value of α and the step size ϕ are very important. A good initial α value can help the algorithm find good individuals faster. A good step size can help α faster converge to the proper value. Thus, we design a set of experiments to identify a proper initial α value and step size. We set 3 different initial α values, i.e., 0, 1 and 100. When initial α value is set to 0, the algorithm starts from the traditional dominance that treats the two objectives equally. When initial α value is set to 1, the algorithm focuses more on the effectiveness. When initial α value is set to 100, the algorithm almost becomes a single-objective algorithm that only optimise the effectiveness. We also have 5 different step size, 10, 20, 30, 50 and 90. Due to the page limit, we summary the experimental results in the boxplot format, each box in the boxplot represents the distribution of the 30 runs of the proposed algorithm with the corresponding parameter setting over the 57 instances. We examined the performance on both HV and IGD. As IGD has the same pattern as the HV, we omit the IGD results here due to the page limit. The results are shown in Fig. 5.

From Fig. 5, we can observe that the algorithms with initial α value of 0 and 1 perform better than algorithms with initial α value of 100 as they can obtain more stable results and which is independent of the θ value. This indicates that a smaller initial value is a better choice. By looking into the results, when α equals 100, the algorithm will focus too much on the effectiveness, and almost becomes a single-objective algorithm. The population is occupied by only large individuals. This makes it hard for the algorithm to find small effective individuals. As a result, it is difficult for the algorithm



(a) α = 0 (b) α = 1 (c) α = 100

Fig. 5. The experimental results for parameter sensitivity analysis in terms of HV.

to find a complete Pareto front. Intuitively, 0 is a better initial value than 1 since α dominance becomes traditional dominance when α equals 0. We can use the traditional dominance as a start point. For the step size, it can be seen that step size is not very sensitive when initial value equals 0 and 1, but 90 seems to be a slightly better choice. Therefore, the initial α value of 0 and step size of 90 are utilised in the subsequent experiments.

C. Results and Discussion

We use the Wilcoxon rank sum test with the significance level of 0.05 for statistical analysis. In each table, each "+", "-" or "=" next to αMOGP-a indicates that αMOGP-a performed statistically significantly better than, worse than, or comparable to the compared algorithm.

1) Hyper-Volume and IGD Test Performance: Table III show the results of the mean and standard deviation of HV and IGD obtained by the compared MOGP methods on the test samples.

From Table III, we can see that αMOGP-a outperforms TSNSGPII-a on 21 out of 58 instances and achieves comparable results on remaining instances in terms of HV. The αMOGP-a also performs significantly better than TSNSGPII-a on 17 out of 58 instances and achieves comparable results on 40 instances in terms of IGD. This is as expected since αMOGP-a can better handle the objective selection bias issue and the issue of potentially good individuals. By looking further into the results, we can also see that even though αMOGP-a and TSNSGPII-a obtain competitive results on many instances, the mean value of each instance of αMOGP-a is slightly better than that achieved by TSNSGPII-a in terms of HV and IGD. All these observations can indicate that αMOGP-a can generate better Pareto front than the TSNSGPII-a on some instances and generate at least competitive Pareto front on other instances.

The Pareto front with the median HV over the 30 independent runs on some representative instances is shown in Fig. 6. It can be seen that the non-dominated solutions

TABLE III
THE MEAN AND STANDARD DEVIATION OF THE **HV** AND **IGD** OF THE COMPARED MOGP METHODS ON THE ON **UGDB**, **UVAL** AND **UHEFEI** DATASET.

	HV		IGD	
Instance	TSNSGPII-a	αMOGP-a	TSNSGPII-a	αMOGP-a
Ugdb1	0.9327(0.015)	0.9459(0.017)(+)	0.0571(0.015)	0.0428(0.012)(+)
Ugdb2	0.9331(0.026)	0.9501(0.015)(+)	0.0618(0.024)	0.0488(0.012)(+)
Ugdb3	0.9648(0.005)	0.9662(0.005)(=)	0.1111(0.024)	0.1350(0.023)(-)
Ugdb4	0.9749(0.018)	0.9755(0.011)(=)	0.0672(0.016)	0.0710(0.016)(=)
Ugdb5	0.9416(0.020)	0.9512(0.016)(+)	0.0569(0.018)	0.0467(0.012)(+)
Ugdb6	0.9146(0.027)	0.9137(0.024)(=)	0.0610(0.017)	0.0597(0.015)(=)
Ugdb7	0.9447(0.006)	0.9531(0.012)(+)	0.0361(0.003)	0.0334(0.007)(+)
Ugdb8	0.9260(0.011)	0.9358(0.012)(+)	0.066(0.0150)	0.0616(0.016)(=)
Ugdb9	0.8962(0.022)	0.9068(0.022)(=)	0.1039(0.022)	0.0990(0.022)(=)
Ugdb10	0.9553(0.014)	0.9600(0.013)(=)	0.0698(0.015)	0.0674(0.013)(=)
Ugdb11	0.8233(0.028)	0.8257(0.021)(=)	0.1321(0.026)	0.1293(0.019)(=)
Ugdb12	0.8835(0.036)	0.9025(0.033)(+)	0.0819(0.030)	0.0636(0.025)(+)
Ugdb13	0.9215(0.019)	0.9307(0.018)(=)	0.0498(0.013)	0.0453(0.014)(=)
Ugdb14	0.9438(0.021)	0.9405(0.014)(=)	0.0467(0.014)	0.0429(0.008)(=)
Ugdb15	0.9827(0.008)	0.9842(0.013)(=)	0.0250(0.009)	0.0216(0.010)(=)
Ugdb16	0.9401(0.004)	0.9392(0.002)(=)	0.0946(0.017)	0.0774(0.016)(+)
Ugdb17	0.9858(0.001)	0.9945(0.004)(+)	0.0218(0.001)	0.0145(0.004)(+)
Ugdb18	0.9606(0.014)	0.9647(0.012)(=)	0.0408(0.013)	0.0366(0.010)(=)
Ugdb19	0.9697(0.036)	0.9811(0.024)(+)	0.0705(0.024)	0.0630(0.016)(=)
Ugdb20	0.9359(0.024)	0.9346(0.026)(=)	0.0493(0.017)	0.0471(0.015)(=)
Ugdb21	0.9305(0.017)	0.9337(0.016)(=)	0.0626(0.015)	0.0581(0.013)(=)
Ugdb22	0.9079(0.030)	0.9281(0.016)(+)	0.0658(0.024)	0.0476(0.014)(+)
Ugdb23	0.9151(0.013)	0.9232(0.012)(+)	0.0639(0.011)	0.0549(0.007)(+)
Uhefei	0.8181(0.043)	0.8924(0.043)(+)	0.1304(0.036)	0.0781(0.03)(+)
Uval1A	0.9772(0.009)	0.9777(0.013)(=)	0.0239(0.006)	0.0218(0.007)(=)
Uval1B	0.9747(0.015)	0.9753(0.009)(=)	0.0215(0.010)	0.0209(0.010)(=)
Uval1C	0.8787(0.031)	0.8910(0.026)(=)	0.0784(0.028)	0.0701(0.021)(=)
Uval2A	0.9723(0.009)	0.9735(0.012)(=)	0.0323(0.006)	0.0281(0.007)(+)
Uval2B	0.9587(0.012)	0.9649(0.012)(+)	0.0372(0.009)	0.0330(0.007)(=)
Uval2C	0.8048(0.049)	0.8289(0.034)(+)	0.1389(0.039)	0.1163(0.024)(+)
Uval3A	0.9673(0.010)	0.9751(0.009)(+)	0.0415(0.006)	0.0357(0.007)(+)
Uval3B	0.8827(0.024)	0.8959(0.020)(+)	0.0878(0.018)	0.0766(0.016)(+)
Uval3C	0.8513(0.047)	0.8521(0.051)(=)	0.1219(0.040)	0.1224(0.044)(=)
Uval4A	0.9673(0.011)	0.9712(0.008)(=)	0.0433(0.012)	0.0466(0.010)(=)
Uval4B	0.9672(0.011)	0.9689(0.013)(=)	0.0382(0.009)	0.0386(0.011)(=)
Uval4C	0.9538(0.013)	0.9628(0.011)(+)	0.0510(0.010)	0.0454(0.010)(+)
Uval4D	0.8814(0.036)	0.8979(0.036)(=)	0.0771(0.023)	0.0685(0.022)(=)
Uval5A	0.9757(0.006)	0.9772(0.002)(=)	0.0253(0.008)	0.0226(0.005)(=)
Uval5B	0.9662(0.009)	0.9728(0.006)(+)	0.0296(0.008)	0.0238(0.007)(+)
Uval5C	0.9643(0.011)	0.9674(0.008)(=)	0.0329(0.010)	0.0285(0.007)(+)
Uval5D	0.9265(0.012)	0.9428(0.015)(+)	0.0778(0.013)	0.0713(0.014)(=)
Uval6A	0.9785(0.006)	0.9782(0.008)(=)	0.0297(0.010)	0.0307(0.012)(=)
Uval6B	0.9644(0.008)	0.9626(0.016)(=)	0.0437(0.010)	0.0424(0.016)(=)
Uval6C	0.9071(0.033)	0.9114(0.040)(=)	0.0726(0.024)	0.0683(0.029)(=)
Uval7A	0.8902(0.035)	0.8821(0.037)(=)	0.0712(0.022)	0.0737(0.024)(=)
Uval7B	0.9399(0.017)	0.9476(0.019)(=)	0.0558(0.009)	0.0493(0.012)(=)
Uval7C	0.9328(0.025)	0.9444(0.025)(+)	0.0556(0.019)	0.0474(0.018)(=)
Uval8A	0.9752(0.006)	0.9789(0.006)(+)	0.0386(0.013)	0.0350(0.013)(=)
Uval8B	0.9673(0.010)	0.9697(0.006)(=)	0.0342(0.009)	0.0331(0.007)(=)
Uval8C	0.9126(0.020)	0.9197(0.015)(=)	0.0776(0.016)	0.0724(0.015)(=)
Uval9A	0.9692(0.007)	0.9713(0.005)(=)	0.0282(0.006)	0.0258(0.006)(=)
Uval9B	0.9646(0.012)	0.9656(0.010)(=)	0.0332(0.009)	0.0365(0.008)(=)
Uval9C	0.9577(0.007)	0.9588(0.011)(=)	0.0301(0.006)	0.0295(0.009)(=)
Uval9D	0.9176(0.012)	0.9211(0.008)(=)	0.0617(0.011)	0.0584(0.010)(=)
Uval10A	0.9765(0.010)	0.9764(0.012)(=)	0.0254(0.008)	0.0254(0.009)(=)
Uval10B	0.9772(0.004)	0.9781(0.005)(=)	0.0481(0.015)	0.0530(0.018)(=)
Uval10C	0.9689(0.008)	0.9707(0.007)(=)	0.0388(0.011)	0.0384(0.011)(=)
Uval10D	0.9643(0.012)	0.9701(0.010)(+)	0.0408(0.010)	0.0377(0.010)(=)

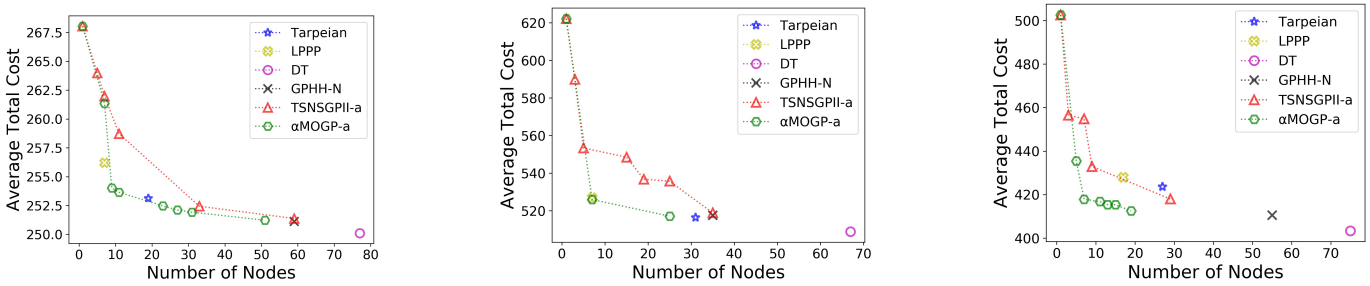
generated by αMOGP-a is better than TSNSGPII-a. Specifically, αMOGP-a can find solutions with better effectiveness than that obtained by TSNSGPII-a. This is because αMOGP-a use the α dominance to adaptively give more pressure to the effectiveness during the evolutionary process when necessary. This can help αMOGP-a generating more effective and compact individuals during the evolutionary process. As other algorithms are single-objective algorithms, their Pareto fronts have only one point. We can see that the solutions of Tarpeian and LPPP are allocated on the left side of each figure. This indicates that Tarpeian AND LPPP evolve small but less effective solutions. The solutions of DT and GPHH-N are allocated on the right side of each figure. This indicates that DT and GPHH-N evolve effective but large solutions. The αMOGP-a can evolve Pareto front which contains solutions varies from small (ineffective) to large (effective). As a result, users can choose a solution based on their preference.

2) Most Effective Individual: Although a set of non-dominated solutions are obtained, only a single individual needs to be selected as the routing policy in practical use. Here, we consider the effectiveness as the primary objective, and select the individual with the best effectiveness from the non-dominated set. Then, we compare the effectiveness and size of the most effective individuals obtained by each algorithm. Each "+", "-" or "=" next to the compared algorithms indicates that αMOGP-a performs statistically significantly better than, worse than, or comparable to the compared algorithm.

Table IV shows the mean and standard deviation of the average total cost (effectiveness) of the compared algorithms. It can be seen that αMOGP-a performs significantly better than Tarpeian on 32 out of 58 instances, and never performs significantly worse than Tarpeian. The αMOGP-a also outperforms LPPP on almost all instances. This is an interesting observation that both DT and GPHH-N can outperform αMOGP-a on a number of instances. DT will first use tournament selection to select multiple effective individuals and then select the smallest one from the selected ones as parent. So DT still prefers to select the effective individuals as parent. For GPHH-N, it utilises the niching simplification method to reduce the size but its main goal is still on the effectiveness. Both algorithms focus more on the effectiveness. Thus, they can achieve better effectiveness than αMOGP-a on some instances. However, if we go a step further and look into the experimental results, we see that the effectiveness of αMOGP-a is just slightly worse than that in DT and GPHH-N. Furthermore, as we will see later, αMOGP-a can obtain much smaller size than DT and GPHH-N. In addition, αMOGP-a can outperform the TSNSGPII-a on 18 instances and never perform significantly worse than TSNSGPII-a on remaining instances.

Table V shows the mean and standard deviation of the average size (number of nodes) of the compared algorithms. We can see that αMOGP-a performs significantly worse than Tarpeian and LPPP. Tarpeian and LPPP can evolve much smaller routing policies than αMOGP-a. However, this size reduction is compromised by sacrificing effectiveness. Tarpeian and LPPP can only evolve small but ineffective routing policies. In contrast, DT and GPHH-N can evolve very effective routing policies, but the size of the evolved routing policies is much larger than αMOGP-a. The αMOGP-a can outperform DT and GPHH-N on almost all instances. It is worth noting that although αMOGP-a and TSNSGPII-a obtain competitive results in terms of size, αMOGP-a outperforms TSNSGPII-a in terms of the effectiveness according to Table IV.

To sum up, from the perspective of both benchmark datasets and real-world dataset, αMOGP-a can evolve better Pareto



(a) Ugdb23 (b) Uval5C (c) Uval6C

Fig. 6. The non-dominated solution of the run with the median HV over 30 runs on compared algorithms.

front than TSNSGPII-a. For the benchmark datasets, αMOGP-a evolves much smaller routing policies by slightly losing effectiveness comparing with DT and GPHH-N. Tarpeian and LPPP can evolve much smaller routing policies than αMOGP-a, but their effectiveness is much worse. For the real-world instance, αMOGP-a evolves much smaller routing policies without losing effectiveness comparing with DT and GPHH-N. It evolves significantly better routing policies in terms of both effectiveness of size than Tarpeian and LPPP. Thus, all the results indicate that αMOGP-a can evolve compact and effective routing policies.

V. Further Analysis

A. Component Analysis

To verify the effectiveness of each component of the newly proposed αMOGP-a, we designed a set of control experiments on some representative instances. We compare αMOGP-a which utilises the new α adaptation scheme with an existing α adaptation scheme proposed in [15]. The existing α adaptation scheme utilises both the effectiveness and size to identify bias and adjust α value by simply +/- 0.1 each time. We also compare αMOGP-a with the counterpart without using the archive (αMOGP). The experimental results are shown in Table VI. As IGD has the same pattern as the HV, we omit the IGD results here due to the page limit. Each "+", "-" or "=" next to the compared algorithms indicates that αMOGP-a performs statistically significantly better than, worse than, or comparable to the compared algorithm.

From Table VI, we can see that αMOGP-a significantly outperforms the one using the existing α adaptation scheme. This indicates the effectiveness of the new α adaptation scheme. In addition, we can see that without using the archive, the performance of αMOGP-a significantly deteriorates. This demonstrates the effectiveness of using the archive.

B. α Adaptation

To further analyse how the new α adaptation scheme outperforms the existing one, we plot the convergence curve of α value on 8 representative instances on 30 independent runs. The convergence curve is shown in Fig. 7. Each dot in the figure is the mean α value over 30 independent runs at the generation.

From Fig. 7a, it can be seen that with the existing α adaptation scheme, α value keeps increasing until the end. This shows that 0.1 is not a proper step size for the α value. In contrast, the convergence curve in Fig. 7b grows rapidly at first, and as generation increases, the curves stabilise and eventually converge. We can see some fluctuations above the convergence curve. This is expected as when α is too large we need to decrease it, while α is too small we need to increase it. In the process of finding the proper α value, the α value will fluctuate around the proper value, and eventually converge to the proper α value. This indicates that the new α adaptation scheme has better capability to find the proper α value. At the same time, Fig. 7b also confirms our hypothesis that different α values are required for different instances.

C. Semantic Analysis of Evolved Policies

We extract an effective routing policy evolved by αMOGP-a on Ugdb6 instance to make some further understanding of the behavior of the GP evolved routing policy. To interpret the routing policy evolved by αMOGP-a, we rewrite it by the following equations,

$$RP = S_1 + S_2 + S_3, \tag{17}$$

where

$$S_1 = \mathsf{CFH}, \tag{18}$$

$$S_2 = (\mathsf{FULL} - 0.55) * \mathsf{CTD}, \tag{19}$$

$$S_3 = \max(\mathsf{CTT1}, \mathsf{DEM1}). \tag{20}$$

We can interpret the above routing policy as follows.

- The policy always prefers tasks close to the current location. This is consistent with the well-known nearest neighbour heuristic.
- If $\mathsf{FULL} \geq 0.55$, i.e., the vehicle is more than half full, the coefficient of CTD in S_2 is non-negative. The policy prefers tasks close to the depot. The focus on CTD increases as the vehicle fills up. This is consistent with our intuition that as the vehicle becomes more full, it is more likely to return to the depot to refill. If the route failure occurs on a task near the depot, the recourse cost is minimised. If $\mathsf{FULL} < 0.55$, the route failure is unlikely to occur. The policy prefers tasks that are distant from the depot, to leave the tasks closer to the depot to the

TABLE IV
THE EXPERIMENTAL RESULTS OF SINGLE-OBJECTIVE INDICATOR (**EFFECTIVENESS**) FOR COMPARED ALGORITHMS ON **UGDB**, **UVAL** AND **UHEFEI** DATASETS.

Instance	Tarpeian [64]	LPPP [64]	DT [64]	GPHH-N [49]	TSNSGPII-a [17]	αMOGP-a
Ugdb1	359.741(9.67)(+)	361.5418(8.43)(+)	353.9708(17.169)(=)	347.0554(5.705)(-)	354.4534(6.151)(+)	349.8263(6.501)
Ugdb2	376.6364(11.098)(+)	385.6952(10.432)(+)	371.489(7.852)(=)	369.7039(7.791)(=)	375.5819(9.299)(+)	369.4091(5.863)
Ugdb3	307.3752(1.659)(=)	311.9899(4.892)(+)	306.3066(2.038)(=)	307.6944(2.444)(+)	307.1549(1.537)(=)	306.8357(1.532)
Ugdb4	324.5011(4.963)(+)	331.275(16.411)(+)	322.1242(1.958)(=)	322.646(1.942)(=)	322.7723(4.693)(=)	322.0611(2.534)
Ugdb5	432.4777(9.581)(=)	451.353(14.359)(+)	425.016(8.723)(=)	422.739(6.281)(-)	433.6248(11.01)(=)	428.6176(8.313)
Ugdb6	362.5787(16.472)(+)	362.7825(1.89)(+)	343.9608(9.035)(-)	338.8735(3.449)(-)	349.2738(8.652)(=)	349.8394(7.416)
Ugdb7	359.4155(0.96)(+)	359.8538(0.0)(+)	357.3026(3.399)(+)	355.1464(4.582)(=)	357.2867(1.504)(+)	354.9296(3.213)
Ugdb8	441.1566(12.024)(+)	456.8996(13.306)(+)	432.0626(10.601)(=)	430.0516(6.924)(=)	437.2732(4.744)(+)	432.5656(5.593)
Ugdb9	399.2624(8.284)(+)	406.644(11.598)(+)	392.9335(10.982)(=)	383.0641(8.999)(-)	396.7845(8.619)(+)	392.4596(8.449)
Ugdb10	296.5628(4.87)(+)	300.0145(2.93)(+)	291.9685(5.712)(=)	289.9431(4.95)(-)	294.8434(3.747)(=)	293.4669(3.696)
Ugdb11	443.5919(5.715)(+)	446.8559(4.755)(+)	438.2663(5.222)(=)	432.2868(8.233)(-)	439.2069(4.559)(=)	438.8171(3.598)
Ugdb12	618.1571(14.697)(+)	624.9459(18.742)(+)	606.285(16.707)(=)	605.8586(9.306)(=)	610.9021(15.611)(+)	604.2255(13.99)
Ugdb13	585.3549(7.584)(=)	593.7929(10.522)(+)	581.0675(7.594)(=)	576.8475(7.093)(-)	585.7524(6.656)(=)	582.4751(6.353)
Ugdb14	108.6919(2.039)(=)	118.0813(2.556)(+)	108.072(2.439)(-)	107.0618(1.437)(-)	108.0788(1.482)(=)	108.4235(1.018)
Ugdb15	58.3416(0.316)(=)	62.0083(0.0)(+)	58.5233(0.974)(=)	58.4673(1.037)(=)	58.2869(0.187)(=)	58.2604(0.356)
Ugdb16	134.5298(0.06)(+)	134.4749(0.0)(=)	134.5552(0.052)(+)	134.4879(0.411)(=)	134.393(0.222)(=)	134.4488(0.14)
Ugdb17	91.2706(0.0)(+)	93.8657(0.0)(+)	91.1897(0.251)(+)	91.2842(0.107)(+)	91.2649(0.03)(+)	91.076(0.091)
Ugdb18	169.0591(2.795)(=)	182.0322(6.18)(+)	168.7085(4.76)(=)	166.973(1.606)(-)	169.0211(2.14)(=)	168.5346(1.795)
Ugdb19	61.7611(1.723)(=)	68.0709(0.802)(+)	61.5426(1.902)(=)	60.8448(0.547)(=)	61.7781(1.601)(=)	61.3057(1.071)
Ugdb20	128.8132(2.175)(=)	137.3079(0.0)(+)	127.914(4.238)(=)	126.8208(2.337)(-)	127.794(1.565)(=)	127.9286(1.693)
Ugdb21	166.2336(3.428)(=)	180.8426(5.53)(+)	165.9424(4.636)(=)	164.6705(3.54)(-)	166.5348(1.982)(=)	166.0947(1.814)
Ugdb22	211.7671(3.45)(+)	221.2032(2.724)(+)	210.1016(2.105)(=)	210.0523(2.478)(=)	210.8277(2.0)(+)	209.5899(1.034)
Ugdb23	251.9495(2.512)(=)	257.2459(4.187)(+)	251.1037(2.753)(=)	248.748(1.801)(-)	251.8207(1.736)(=)	251.0229(1.492)
Uhefei	322082(7441)(+)	317139(7605)(+)	316545(7008)(=)	311694(7202)(=)	327762(7131)(+)	314013(6997)
Uval1A	181.9436(4.437)(+)	188.8427(6.633)(+)	177.7145(5.283)(=)	174.5455(1.57)(-)	178.3407(2.691)(=)	178.2425(3.673)
Uval1B	186.0304(2.533)(=)	190.4595(3.237)(+)	183.8868(1.058)(-)	183.7689(0.869)(-)	186.0899(3.287)(=)	185.6556(1.88)
Uval1C	320.5283(10.524)(+)	329.6197(8.438)(+)	315.8533(8.309)(=)	309.8156(12.435)(-)	317.9679(8.34)(=)	314.4565(7.278)
Uval2A	233.3371(6.258)(=)	239.207(5.083)(+)	231.5573(4.049)(=)	230.5274(3.451)(-)	232.9362(2.52)(=)	232.7639(3.327)
Uval2B	283.4977(12.472)(+)	295.0745(15.143)(+)	279.484(3.985)(=)	278.0283(3.801)(=)	280.5628(4.185)(=)	278.6683(4.082)
Uval2C	617.0144(18.002)(+)	622.1776(13.246)(+)	598.0613(22.492)(=)	591.1879(14.467)(-)	615.4274(21.033)(+)	606.9756(14.595)
Uval3A	82.8063(1.183)(=)	88.3126(2.176)(+)	82.4896(1.249)(=)	81.7668(0.698)(-)	83.3861(0.956)(+)	82.5919(0.906)
Uval3B	100.1533(3.669)(+)	101.8713(2.175)(+)	96.5034(1.66)(=)	95.3129(1.293)(-)	97.4254(1.338)(=)	96.8018(1.08)
Uval3C	179.7858(9.375)(=)	193.8458(14.893)(+)	177.4681(5.522)(-)	173.7863(4.013)(-)	182.4238(6.7)(=)	182.3861(7.16)
Uval4A	430.4099(27.157)(+)	429.4005(7.934)(+)	420.1009(5.788)(=)	417.5135(4.281)(-)	424.1895(6.985)(=)	421.1683(4.948)
Uval4B	448.8171(10.208)(=)	457.2661(12.784)(+)	444.0003(8.243)(=)	439.2253(6.023)(-)	446.7894(7.41)(=)	445.7183(9.372)
Uval4C	499.4249(12.378)(+)	503.3278(13.547)(+)	493.0886(17.139)(=)	486.7424(11.729)(-)	497.6095(10.661)(+)	490.4748(8.96)
Uval4D	726.6945(34.562)(=)	728.593(39.509)(=)	701.2216(31.662)(=)	689.6775(22.469)(-)	722.9498(28.646)(=)	709.5947(28.792)
Uval5A	444.9818(4.721)(+)	451.2758(3.761)(+)	440.4567(3.934)(-)	440.6779(8.742)(-)	443.9832(4.71)(=)	442.8144(2.026)
Uval5B	477.6904(8.99)(+)	479.6288(7.16)(+)	475.9497(27.624)(=)	467.0353(4.985)(-)	476.082(6.158)(+)	471.7457(3.856)
Uval5C	522.0263(6.645)(+)	530.2507(12.634)(+)	518.6073(8.35)(=)	513.4305(6.425)(=)	518.4106(7.573)(=)	516.0506(6.337)
Uval5D	747.1994(20.341)(+)	751.3212(20.016)(+)	728.8122(13.066)(=)	725.5403(15.047)(=)	735.3973(10.431)(+)	722.9408(11.808)
Uval6A	229.8105(2.513)(=)	232.7176(4.722)(+)	229.3166(2.951)(=)	228.1973(1.736)(=)	229.7946(2.232)(=)	229.4904(2.935)
Uval6B	260.987(4.221)(+)	268.2549(10.851)(+)	257.3596(4.19)(=)	256.8229(4.516)(=)	258.3724(3.261)(=)	259.3178(6.059)
Uval6C	423.3921(21.584)(=)	431.228(22.874)(+)	405.4078(8.073)(-)	401.385(6.803)(-)	416.9153(14.703)(=)	415.2576(17.743)
Uval7A	291.7678(6.221)(=)	300.7732(7.065)(+)	289.6651(10.374)(-)	285.0704(4.362)(-)	291.7044(5.248)(=)	292.8277(5.272)
Uval7B	299.9158(8.327)(=)	310.1024(9.85)(+)	294.6197(6.177)(-)	291.9621(6.751)(-)	304.7538(7.55)(=)	301.0962(7.967)
Uval7C	417.0744(17.023)(+)	413.6364(10.863)(=)	407.9503(8.14)(=)	406.1523(4.853)(=)	416.4156(12.029)(+)	411.1995(12.162)
Uval8A	398.8698(3.124)(+)	404.9157(5.031)(+)	398.6413(3.961)(=)	400.2917(12.315)(=)	400.0483(3.017)(+)	397.5535(2.686)
Uval8B	435.4858(14.72)(+)	438.6551(8.929)(+)	428.9821(9.303)(=)	424.48(5.044)(-)	429.9672(6.382)(=)	428.3036(4.022)
Uval8C	689.1992(14.948)(+)	690.6813(14.953)(+)	666.4134(18.001)(-)	661.1327(14.932)(-)	677.2028(13.883)(=)	672.4655(10.534)
Uval9A	336.7522(3.242)(=)	343.2224(7.196)(+)	334.4052(2.242)(-)	332.6537(2.86)(-)	337.6159(3.129)(+)	336.4662(2.551)
Uval9B	351.4968(3.71)(=)	356.7118(5.774)(+)	350.7538(3.445)(=)	347.8707(3.993)(=)	350.8984(6.068)(=)	350.211(5.154)
Uval9C	369.8666(5.713)(+)	375.803(4.828)(+)	363.2856(5.335)(=)	362.1799(4.098)(-)	367.4177(4.386)(=)	366.1998(6.142)
Uval9D	490.5418(13.782)(+)	492.4926(10.905)(+)	480.2601(10.236)(=)	474.6573(12.982)(-)	483.0787(8.324)(=)	480.3587(6.0)
Uval10A	444.3772(10.897)(=)	461.3147(22.035)(+)	439.147(4.092)(-)	437.5081(2.54)(-)	444.0889(5.229)(=)	444.2145(6.384)
Uval10B	462.9993(6.095)(=)	471.7224(17.878)(+)	460.3632(5.944)(=)	456.189(4.304)(-)	462.8599(3.279)(=)	461.9855(4.145)
Uval10C	483.9717(5.967)(=)	489.8595(11.404)(+)	480.028(6.693)(=)	476.3406(6.219)(-)	482.8358(6.115)(=)	481.3485(5.005)
Uval10D	622.7074(7.153)(=)	624.3295(7.891)(+)	619.4135(4.145)(=)	618.6131(5.903)(=)	625.817(10.896)(+)	620.4912(8.373)

states where the route failure is more likely to occur (the vehicle is more full).

- If $\mathsf{CTT1} \geq \mathsf{DEM1}$, $S_3 = \mathsf{CTT1}$. In other words, if the cost from the candidate task to its nearest task is greater than the demand of the candidate task's nearest task, the policy tends to select tasks that are more isolated to the other remaining tasks. On the other hand, if $\mathsf{CTT1} < \mathsf{DEM1}$, then $S_3 = \mathsf{DEM1}$. In other words, if the candidate task is close to some other remaining tasks (i.e., the cost is smaller than the demand of the other remaining tasks), then the policy prefers tasks whose nearby remaining tasks have smaller demands. This way, after serving the candidate task, the vehicle can easily serve its nearby remaining task with low cost and probability of route failure (due to the small demand of the nearby task).

By observing the Pareto front found by αMOGP-a, we have

TABLE V
THE EXPERIMENTAL RESULTS OF SINGLE-OBJECTIVE INDICATOR (**SIZE**) FOR COMPARED ALGORITHMS ON **UGDB**, **UVAL** AND **UHEFEI** DATASETS.

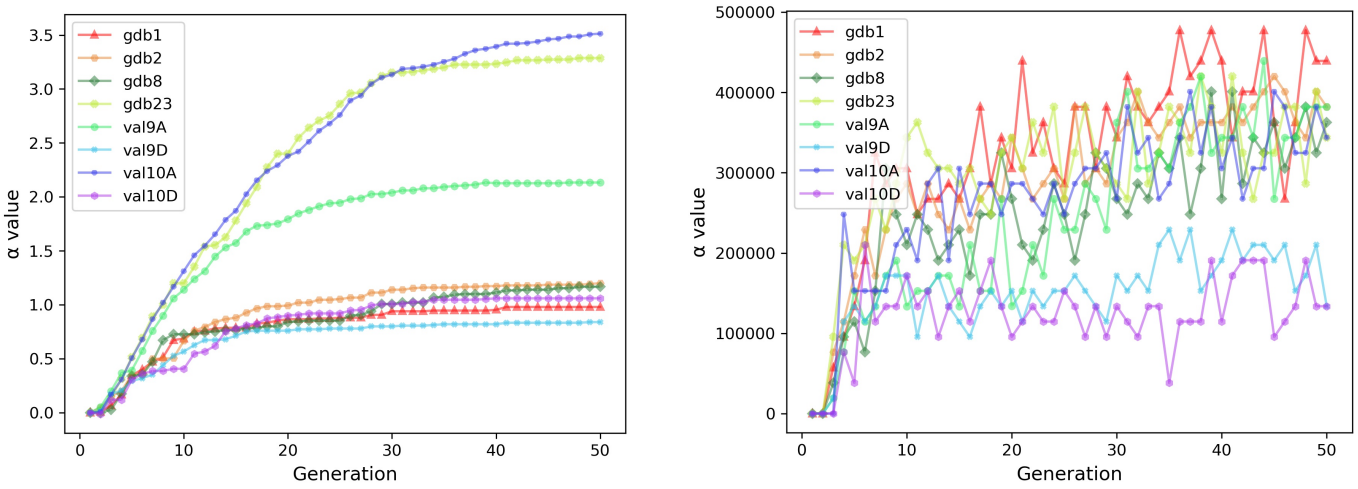
Instance	Tarpeian [64]	LPPP [64]	DT [64]	GPHH-N [49]	TSNSGPII-a [17]	αMOGP-a
Ugdb1	21.7333(13.24)(-)	7.2667(3.095)(-)	84.9333(22.416)(+)	60.6(24.732)(+)	38.8(32.65)(=)	37.0(32.415)
Ugdb2	26.5333(11.407)(=)	9.1333(4.783)(-)	80.8667(29.172)(+)	52.1333(15.037)(+)	32.8(19.975)(=)	35.4667(19.929)
Ugdb3	19.4(8.589)(=)	7.5333(2.097)(-)	68.8(22.562)(+)	43.2667(23.511)(+)	26.8(17.47)(+)	18.2667(10.339)
Ugdb4	22.4(15.078)(=)	7.9333(3.352)(-)	62.0(22.935)(+)	55.0667(32.477)(+)	23.1333(18.827)(=)	24.5333(20.981)
Ugdb5	24.0667(10.837)(-)	8.4(5.096)(-)	77.1333(20.764)(+)	49.6(22.469)(+)	29.8(25.296)(-)	36.6667(26.57)
Ugdb6	12.0667(17.599)(-)	4.1333(5.138)(-)	62.6667(23.265)(+)	40.6667(17.905)(+)	20.2(11.454)(=)	18.8667(12.193)
Ugdb7	4.9333(10.069)(-)	1.0(0.0)(-)	41.2667(19.998)(+)	25.5333(12.616)(+)	10.3333(6.546)(-)	20.3333(21.22)
Ugdb8	31.2(16.949)(=)	8.4667(4.2)(-)	73.7333(29.231)(+)	60.1333(17.322)(+)	42.7333(35.544)(=)	43.7333(33.398)
Ugdb9	29.0(20.477)(=)	8.6667(3.155)(-)	62.8(20.317)(+)	61.4667(15.498)(+)	26.9333(18.274)(=)	26.3333(10.807)
Ugdb10	15.2667(8.737)(-)	5.0667(0.365)(-)	55.6667(21.575)(+)	29.3333(16.66)(=)	16.4667(12.789)(-)	25.5333(19.75)
Ugdb11	18.4(19.098)(=)	1.8667(2.609)(-)	55.4667(22.759)(+)	41.2(11.854)(+)	16.9333(15.461)(=)	17.4667(12.292)
Ugdb12	26.4(11.77)(=)	14.0667(4.571)(-)	80.2(26.343)(+)	53.0(16.526)(+)	36.0(21.047)(=)	28.6667(11.698)
Ugdb13	23.2667(10.657)(=)	7.2667(2.333)(-)	62.8(20.774)(+)	54.0(24.148)(+)	26.4(16.569)(=)	26.6667(15.296)
Ugdb14	23.5333(9.38)(=)	1.4(1.221)(-)	70.7333(21.053)(+)	46.9333(23.368)(+)	30.0(14.218)(=)	23.6667(8.949)
Ugdb15	25.8667(15.71)(=)	1.0(0.0)(-)	89.8667(37.522)(+)	32.8(25.877)(=)	18.2(9.974)(-)	33.2(24.57)
Ugdb16	2.5333(1.634)(+)	1.0(0.0)(=)	19.4667(18.35)(+)	9.0667(15.478)(+)	3.3333(7.485)(=)	1.8(3.221)
Ugdb17	4.0667(1.363)(-)	1.0(0.0)(-)	94.6667(49.248)(+)	6.8(13.166)(-)	5.2(9.253)(-)	21.3333(21.381)
Ugdb18	32.4667(16.358)(=)	5.2(2.987)(-)	83.5333(37.12)(+)	57.4667(31.191)(+)	36.8667(22.966)(=)	29.8667(15.22)
Ugdb19	16.1333(5.625)(+)	1.2(0.61)(-)	81.2(32.464)(+)	16.2667(14.888)(=)	15.5333(12.897)(=)	13.7333(13.781)
Ugdb20	25.1333(16.559)(=)	1.0(0.0)(-)	79.1333(41.556)(+)	43.4(22.963)(+)	24.6667(13.986)(=)	21.2667(9.318)
Ugdb21	25.3333(13.586)(=)	3.2(2.797)(-)	76.2667(33.307)(+)	58.4667(27.827)(+)	28.6(13.87)(=)	30.7333(14.713)
Ugdb22	21.9333(10.356)(-)	1.5333(1.383)(-)	65.9333(25.438)(+)	53.2(23.835)(+)	32.6(20.807)(=)	35.3333(22.561)
Ugdb23	28.4(12.286)(=)	6.1333(1.634)(-)	78.0(23.895)(+)	76.5333(34.894)(+)	38.2(18.108)(=)	29.9333(11.552)
Uhefei	33.6(15.422)(=)	60.8(30.994)(+)	66.1333(20.367)(+)	54.0667(14.816)(+)	29.0667(16.03)(=)	33.1(16.0)
Uval1A	15.6(11.193)(-)	3.8667(2.713)(-)	73.1333(25.249)(+)	30.5333(20.238)(=)	22.6(14.464)(=)	23.6(14.764)
Uval1B	19.9333(10.113)(=)	6.6(2.43)(-)	61.1333(22.505)(+)	26.4667(16.358)(=)	19.8(13.01)(=)	22.8667(15.429)
Uval1C	25.1333(11.708)(=)	9.6(4.207)(-)	75.7333(34.519)(+)	64.9333(20.854)(+)	33.1333(28.657)(=)	31.2667(16.789)
Uval2A	25.2667(12.426)(+)	7.1333(1.655)(-)	72.3333(29.405)(+)	26.5333(15.471)(+)	22.7333(16.161)(=)	17.8667(9.138)
Uval2B	24.6667(13.184)(=)	7.4667(4.289)(-)	74.2667(31.573)(+)	48.2667(27.884)(+)	33.4(16.917)(=)	28.2667(17.265)
Uval2C	26.7333(11.741)(=)	13.8(5.573)(-)	67.1333(24.166)(+)	71.9333(22.245)(+)	41.6(30.477)(=)	33.2(16.055)
Uval3A	28.8667(13.024)(=)	5.6(1.192)(-)	73.1333(28.796)(+)	36.0(15.972)(+)	26.7333(14.881)(=)	26.4(10.906)
Uval3B	14.6667(18.263)(-)	3.0(0.743)(-)	67.5333(24.624)(+)	50.4667(19.355)(+)	23.1333(10.345)(=)	21.2(9.618)
Uval3C	26.2(12.041)(=)	7.9333(3.14)(-)	76.8(37.3)(+)	65.6667(21.867)(+)	30.8(17.697)(=)	32.0(15.148)
Uval4A	25.2(9.357)(=)	8.7333(2.664)(-)	66.4(32.594)(+)	47.3333(19.526)(+)	31.2(14.644)(=)	27.8667(14.352)
Uval4B	28.2(12.743)(=)	12.4667(5.894)(-)	66.2667(23.297)(+)	47.4667(16.698)(+)	30.5333(15.247)(=)	25.0(10.544)
Uval4C	28.2(17.893)(=)	13.4667(3.665)(-)	73.5333(31.015)(+)	56.8(28.218)(+)	35.4667(20.623)(=)	30.9333(12.86)
Uval4D	28.0667(14.477)(-)	18.6667(10.131)(-)	67.6667(18.867)(+)	68.2667(23.893)(+)	29.1333(13.846)(=)	37.0(15.438)
Uval5A	26.8667(17.834)(=)	6.2667(1.437)(-)	59.6667(15.856)(+)	44.9333(16.613)(+)	28.3333(15.654)(=)	24.4(12.353)
Uval5B	30.1333(16.54)(=)	10.8667(3.235)(-)	75.0667(33.336)(+)	46.8667(19.333)(+)	30.5333(19.839)(=)	30.7333(14.732)
Uval5C	19.5333(7.947)(-)	9.3333(4.816)(-)	58.6667(20.186)(+)	41.4667(13.418)(+)	26.2667(18.175)(=)	25.0667(11.441)
Uval5D	20.9333(12.323)(-)	13.2(6.288)(-)	64.6667(22.61)(+)	63.6(19.842)(+)	31.4667(13.602)(=)	27.2(13.435)
Uval6A	23.0(12.171)(=)	9.5333(3.441)(-)	61.5333(29.389)(+)	39.0(21.953)(+)	36.6667(18.952)(+)	25.2(11.559)
Uval6B	23.6(11.205)(=)	9.8(3.773)(-)	68.8(21.88)(+)	48.6(18.977)(+)	34.2(35.929)(=)	30.2(17.377)
Uval6C	29.4(15.725)(=)	9.6667(5.809)(-)	70.0(20.408)(+)	62.6(16.336)(+)	28.4(11.425)(=)	28.5333(14.294)
Uval7A	27.3333(14.726)(=)	9.5333(2.569)(-)	65.0667(27.505)(+)	36.1333(15.046)(+)	27.3333(16.138)(=)	25.7333(10.392)
Uval7B	29.2(13.291)(=)	10.1333(3.739)(-)	81.6(31.488)(+)	38.0(18.411)(+)	33.1333(17.035)(=)	27.9333(11.552)
Uval7C	26.8667(11.59)(-)	14.0667(4.571)(-)	70.7333(25.006)(+)	58.4(19.849)(+)	42.6667(29.758)(=)	38.4(23.764)
Uval8A	24.9333(11.283)(=)	7.4(2.253)(-)	63.4667(16.739)(+)	39.0667(19.415)(+)	24.2(13.783)(=)	27.6667(14.126)
Uval8B	22.4(15.714)(=)	6.5333(2.662)(-)	57.6667(18.257)(+)	41.2667(21.027)(+)	30.5333(17.963)(=)	24.1333(10.887)
Uval8C	27.3333(13.392)(-)	15.0(5.872)(-)	62.6667(22.792)(+)	59.4667(23.286)(+)	36.0(16.54)(=)	36.6(20.358)
Uval9A	21.8667(8.593)(=)	8.1333(2.956)(-)	64.0(27.361)(+)	46.0(21.112)(+)	26.4667(14.659)(=)	21.4667(11.646)
Uval9B	26.0(11.951)(=)	8.0(2.716)(-)	62.4(22.653)(+)	49.4(18.491)(+)	32.8(14.339)(+)	29.0(19.476)
Uval9C	22.4667(10.814)(-)	8.0667(3.778)(-)	75.6(38.129)(+)	53.6667(17.736)(+)	28.4(16.147)(=)	31.7333(13.98)
Uval9D	31.4(11.196)(=)	16.2(4.055)(-)	66.4(20.077)(+)	59.8667(16.1)(+)	47.9333(38.445)(=)	38.0667(15.911)
Uval10A	24.9333(12.935)(=)	5.6(3.529)(-)	62.4(20.545)(+)	31.4(19.035)(+)	23.9333(12.722)(=)	22.2(13.01)
Uval10B	23.2667(12.089)(=)	8.4(3.682)(-)	61.4667(18.463)(+)	50.6(16.141)(+)	21.6667(10.984)(=)	24.0(18.471)
Uval10C	25.4(15.035)(=)	8.3333(3.032)(-)	62.6(16.587)(+)	52.0(13.884)(+)	23.9333(10.329)(=)	26.8(14.681)
Uval10D	27.9333(12.202)(-)	16.6667(5.122)(-)	77.9333(33.362)(+)	66.8(18.029)(+)	36.1333(20.928)(=)	34.9333(12.12)

some other interesting observations. Eqs. (21) and (22) shows two non-dominated routing policies from same Pareto front of a single run on Ugdb6.

$$RP_1 = \mathsf{CFH} + \mathsf{CTT1} * \mathsf{FULL} + \frac{\mathsf{RQ}}{\mathsf{CTD}}, \tag{21}$$

$$RP_2 = \mathsf{CFH} + \mathsf{CTT1} * \mathsf{FULL}. \tag{22}$$

The mean total cost of RP_1 is 351.02, and the mean total cost of RP_2 is 362.75. It can be seen that the only difference between RP_1 and RP_2 is the term $\frac{\mathsf{RQ}}{\mathsf{CTD}}$, but their effectiveness is much different. This indicates that $\frac{\mathsf{RQ}}{\mathsf{CTD}}$ is a useful building block. We analyse its behavior. RQ refers to the remaining capacity of the vehicle and CTD refers to the cost from the candidate task to the depot. We can transform $\frac{\mathsf{RQ}}{\mathsf{CTD}}$ to $\mathsf{RQ} * \frac{1}{\mathsf{CTD}}$. At each decision point, RQ is a constant for all the candidate tasks. Then, CTD becomes the only variable. Obviously, $\frac{1}{\mathsf{CTD}}$ decreases as CTD increases. Thus, $\frac{\mathsf{RQ}}{\mathsf{CTD}}$ tends to select tasks that are distant from the depot. Then, we analyse



(a) Old α adaptation scheme

(b) New α adaptation scheme

Fig. 7. The Convergence Curve of α value over old adaptation scheme and new adaptation scheme on Representative Instances.

TABLE VI
THE **HV** OF THE COMPARED ALGORITHMS IN THE CONTROL EXPERIMENT OF α ADAPTION SCHEME AND ARCHIVE.

Instance	αMOGP-a	αMOGP-a-+/- 0.1	αMOGP
Ugdb1	0.94(0.017)	0.93(0.014)(+)	0.88(0.024)(+)
Ugdb2	0.95(0.015)	0.92(0.018)(+)	0.88(0.029)(+)
Ugdb8	0.93(0.012)	0.92(0.012)(+)	0.84(0.071)(+)
Ugdb23	0.92(0.012)	0.89(0.020)(+)	0.84(0.052)(+)
Uval9A	0.97(0.005)	0.96(0.007)(+)	0.92(0.025)(+)
Uval9D	0.92(0.008)	0.91(0.011)(+)	0.84(0.043)(+)
Uval10A	0.97(0.012)	0.96(0.009)(+)	0.93(0.021)(+)
Uval10D	0.97(0.010)	0.96(0.008)(+)	0.90(0.046)(+)

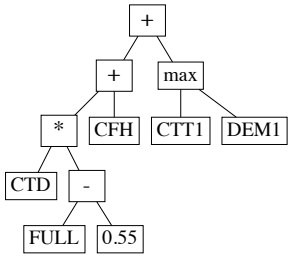


Fig. 8. A Routing Policies Evolved by αMOGP-a.

the impact of RQ. At the beginning of a vehicle route, its remaining capacity of the vehicle is the largest (equal to its capacity). As the service continues, RQ becomes smaller and smaller. This indicates that the impact of $\frac{\text{RQ}}{\text{CTD}}$ decreases when the vehicle serves more tasks and becomes more full. This is similar to the observation in Eq. (19), except that RP_1 does not prefer tasks near the depot when the vehicle is more full. As a result, RP_1 still performs worse than RP shown in Eq. (17).

VI. CONCLUSIONS

The main goal of this work was to evolve both effective and small routing policies. This goal has been successfully achieved by proposing a new MOGP algorithm that utilises the α dominance and the archive (αMOGP-a). The experimental results showed that αMOGP-a can outperform the state-of-the-art algorithms for UCARP. The results showed that both the α adaptation scheme and the archive. are effective. The HV and IGD become worse when either component is removed. We further analysed on several evolved routing policies reveals that CFH is a very important feature, in addition, The importance of the CTD and CTT1 are somewhat related to how fully loaded the vehicle is. Because the new αMOGP-a can provide a set of non-dominated solutions, we can gain some domain knowledge that which building blocks are more useful for routing policy by observing these solutions in the same set of non-dominated solutions. Overall, αMOGP-a can obtain better Pareto front and potentially more interpretable routing policies than the state-of-the-art algorithms.

In the future, we will consider proposing more effective ways that can improve the interpretability of evolved routing policies, such as grammar-based Genetic Programming. We will also try to use the post-hoc methods, such as local explanation and visualisation, to increase the interpretability of the GP-evolved routing policies for UCARP. Last but not least, the new components in αMOGP-a are problem independent. Thus, αMOGP-a is expected to be easily generalised to other uncertain routing problems. We will verify this in our future work.

REFERENCES

[1] Y. Mei, K. Tang, and X. Yao, "Capacitated arc routing problem in uncertain environments," in *Proc. of CEC*. IEEE, 2010, pp. 1–8.
[2] B. L. Golden and R. T. Wong, "Capacitated arc routing problems," *Networks*, vol. 11, no. 3, pp. 305–315, 1981.
[3] S. Wøhlk, "A decade of capacitated arc routing," in *The vehicle routing problem: latest advances and new challenges*. Springer, 2008, pp. 29–48.

[4] U. Ritzinger, J. Puchinger, and R. F. Hartl, "A survey on dynamic and stochastic vehicle routing problems," *International Journal of Production Research*, vol. 54, no. 1, pp. 215–231, 2016.

[5] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proc. of GECCO*. ACM, 2017, pp. 290–297.

[6] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "An improved genetic programming hyper-heuristic for the uncertain capacitated arc routing problem," in *Proc. of AJCAI*. Springer, 2018, pp. 432–444.

[7] Y. Mei and M. Zhang, "Genetic programming hyper-heuristic for multi-vehicle uncertain capacitated arc routing problem," in *Proc. of GECCO*. ACM, 2018, pp. 141–142.

[8] E. D. De Jong, R. A. Watson, and J. B. Pollack, "Reducing bloat and promoting diversity using multi-objective methods," in *Proc. of GECCO*. Morgan Kaufmann Publishers Inc., 2001, pp. 11–18.

[9] R. Poli, N. F. McPhee, and L. Vanneschi, "Elitism reduces bloat in genetic programming," in *Proc. of GECCO*. Citeseer, 2008, pp. 1343–1344.

[10] R. Poli, W. B. Langdon, and N. F. McPhee, "A field guide to genetic programming," *Published via http://lulu. com and freely available at http://www. gp-field-guide. org. uk.(With contributions by JR Koza)*, 2008.

[11] S. Wang, Y. Mei, and M. Zhang, "Novel ensemble genetic programming hyper-heuristics for uncertain capacitated arc routing problem," in *Proc. of GECCO*, 2019, pp. 1093–1101.

[12] J. R. Koza and J. R. Koza, *Genetic programming: on the programming of computers by means of natural selection*. MIT press, 1992.

[13] D. Kinzett, M. Johnston, and M. Zhang, "Numerical simplification for bloat control and analysis of building blocks in genetic programming," *Evolutionary Intelligence*, vol. 2, no. 4, p. 151, 2009.

[14] S. Bleuler, M. Brack, L. Thiele, and E. Zitzler, "Multiobjective genetic programming: Reducing bloat using spea2," in *Proc. of CEC*. IEEE, 2001, pp. 536–543.

[15] S. Wang, Y. Mei, and M. Zhang, "A multi-objective genetic programming approach with self-adaptive α dominance to uncertain capacitated arc routing problem," in *Proc. of CEC*. IEEE, 2021, pp. 636–643.

[16] J. Branke, S. Nguyen, C. W. Pickardt, and M. Zhang, "Automated design of production scheduling heuristics: A review," *IEEE Trans. Evol. Comput.*, vol. 20, no. 1, pp. 110–124, 2015.

[17] S. Wang, Y. Mei, and M. Zhang, "Two-stage multi-objective genetic programming with archive for uncertain capacitated arc routing problem," in *Proc. of GECCO*, 2021, pp. 287–295.

[18] E. W. Dijkstra *et al.*, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[19] K. Ikeda, H. Kita, and S. Kobayashi, "Failure of pareto-based moeas: Does non-dominated really mean near to optimal?" in *Proc. of CEC*, vol. 2. IEEE, 2001, pp. 957–962.

[20] J. M. Belenguer and E. Benavent, "A cutting plane algorithm for the capacitated arc routing problem," *Computers & Operations Research*, vol. 30, no. 5, pp. 705–728, 2003.

[21] B. L. Golden, J. S. DeArmon, and E. K. Baker, "Computational experiments with algorithms for a class of routing problems," *Computers & Operations Research*, vol. 10, no. 1, pp. 47–59, 1983.

[22] G. Ulusoy, "The fleet size and mix problem for capacitated arc routing," *European Journal of Operational Research*, vol. 22, no. 3, pp. 329–337, 1985.

[23] A. Hertz, G. Laporte, and M. Mittaz, "A tabu search heuristic for the capacitated arc routing problem," *Operations research*, vol. 48, no. 1, pp. 129–135, 2000.

[24] J. Brandão and R. Eglese, "A deterministic tabu search algorithm for the capacitated arc routing problem," *Computers & Operations Research*, vol. 35, no. 4, pp. 1112–1126, 2008.

[25] P. Lacomme, C. Prins, and W. Ramdane-Chérif, "A genetic algorithm for the capacitated arc routing problem and its extensions," in *Workshops on Applications of Evolutionary Computation*. Springer, 2001, pp. 473–483.

[26] P. Lacomme, C. Prins, and W. Ramdane-Cherif, "Competitive memetic algorithms for arc routing problems," *Annals of Operations Research*, vol. 131, no. 1-4, pp. 159–185, 2004.

[27] K. Tang, Y. Mei, and X. Yao, "Memetic algorithm with extended neighborhood search for capacitated arc routing problems," *IEEE Trans. Evol. Comput.*, vol. 13, no. 5, pp. 1151–1166, 2009.

[28] Y. Mei, K. Tang, and X. Yao, "Decomposition-based memetic algorithm for multiobjective capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 15, no. 2, pp. 151–165, 2011.

[29] K. F. Doerner, R. F. Hartl, V. Maniezzo, and M. Reimann, "Applying ant colony optimization to the capacitated arc routing problem," in *International Workshop on Ant Colony Optimization and Swarm Intelligence*. Springer, 2004, pp. 420–421.

[30] L.-N. Xing, P. Rohlfshagen, Y.-W. Chen, and X. Yao, "A hybrid ant colony optimization algorithm for the extended capacitated arc routing problem," *IEEE Trans. Syst. Man Cybern.*, vol. 41, no. 4, pp. 1110–1123, 2011.

[31] Y.-H. Jia, Y. Mei, and M. Zhang, "A bilevel ant colony optimization algorithm for capacitated electric vehicle routing problem," *IEEE Trans. Cybern.*, 2021.

[32] L. Feng, Y.-S. Ong, I. W.-H. Tsang, and A.-H. Tan, "An evolutionary search paradigm that learns with past experiences," in *Proc. of CEC*. IEEE, 2012, pp. 1–8.

[33] L. Feng, Y.-S. Ong, A.-H. Tan, and I. W. Tsang, "Memes as building blocks: a case study on evolutionary optimization+ transfer learning for routing problems," *Memetic Computing*, vol. 7, no. 3, pp. 159–180, 2015.

[34] J. Liu, K. Tang, and X. Yao, "Robust optimization in uncertain capacitated arc routing problems: Progresses and perspectives," *IEEE Computational Intelligence Magazine*, vol. 16, no. 1, pp. 63–82, 2021.

[35] G. Fleury, P. Lacomme, C. Prins, and W. Ramdane-Chérif, "Improving robustness of solutions to arc routing problems," *Journal of the operational research society*, vol. 56, no. 5, pp. 526–538, 2005.

[36] J. Wang, K. Tang, and X. Yao, "A memetic algorithm for uncertain capacitated arc routing problems," in *IEEE Workshop on MC*. IEEE, 2013, pp. 72–79.

[37] E. Babaee Tirkolaee, M. Alinaghian, M. Bakhshi Sasi, and M. Seyyed Esfahani, "Solving a robust capacitated arc routing problem using a hybrid simulated annealing algorithm: a waste collection application," *Journal of Industrial Engineering and Management Studies*, vol. 3, no. 1, pp. 61–76, 2016.

[38] J. Wang, K. Tang, J. A. Lozano, and X. Yao, "Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems," *IEEE Trans. Evol. Comput.*, vol. 20, no. 1, pp. 96–109, 2016.

[39] S. Tsutsui, M. Pelikan, and D. E. Goldberg, "Using edge histogram models to solve permutation problems with probabilistic model-building genetic algorithms," *IlliGAL Report*, vol. 2003022, 2003.

[40] H. Tong, L. L. Minku, S. Menzel, B. Sendhoff, and X. Yao, "A novel generalised meta-heuristic framework for dynamic capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2022.

[41] H. Tong, L. Minku, S. Menzel, B. Sendhoff, and X. Yao, "A hybrid local search framework for the dynamic capacitated arc routing problem," in *Proc. of GECCO*, 2021.

[42] T. Weise, A. Devert, and K. Tang, "A developmental solution to (dynamic) capacitated arc routing problems using genetic programming," in *Proc. of GECCO*. ACM, 2012, pp. 831–838.

[43] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "An improved genetic programming hyper-heuristic for the uncertain capacitated arc routing problem," in *AJCAI*. Springer, 2018, pp. 432–444.

[44] ——, "Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems," *Evolutionary computation*, pp. 1–29, 2019.

[45] M. A. Ardeh, Y. Mei, and M. Zhang, "Genetic programming with knowledge transfer and guided search for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, 2021.

[46] M. A. Ardeh, Y. Mei, M. Zhang, and X. Yao, "Knowledge transfer genetic programming with auxiliary population for solving uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, 2022.

[47] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem," *Evolutionary computation*, vol. 28, no. 2, pp. 289–316, 2020.

[48] S. Wang, Y. Mei, J. Park, and M. Zhang, "Evolving ensembles of routing policies using genetic programming for uncertain capacitated arc routing problem," in *Proc. of SSCI*. IEEE, 2019, pp. 1628–1635.

[49] S. Wang, Y. Mei, M. Zhang, and X. Yao, "Genetic programming with niching for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 26, no. 1, pp. 73–87, feb 2022.

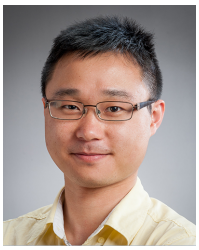
[50] Q. Chen, B. Xue, and M. Zhang, "Rademacher complexity for enhancing the generalization of genetic programming for symbolic regression," *IEEE Trans. Cybern.*, 2020.

[51] D. C. Hooper and N. S. Flann, "Improving the accuracy and robustness of genetic programming through expression simplification," in *Proc. of the conference on genetic programming*, 1996, pp. 428–428.

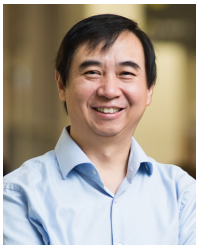
[52] M. Zhang and P. Wong, "Genetic programming for medical classification: a program simplification approach," *Genetic Programming and Evolvable Machines*, vol. 9, no. 3, pp. 229–255, 2008.
[53] A. Song, D. Chen, and M. Zhang, "Contribution based bloat control in genetic programming," in *Proc. of CEC*. IEEE, 2010, pp. 1–8.
[54] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan, "A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii," in *Proc. of PPSN*. Springer, 2000, pp. 849–858.
[55] Q. Zhang and H. Li, "Moea/d: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, 2007.
[56] H. Zhao, "A multi-objective genetic programming approach to developing pareto optimal decision trees," *Decision Support Systems*, vol. 43, no. 3, pp. 809–826, 2007.
[57] A. Cano, S. Ventura, and K. J. Cios, "Multi-objective genetic programming for feature extraction and data visualization," *Soft Computing*, vol. 21, no. 8, pp. 2069–2089, 2017.
[58] S. Nguyen, M. Zhang, M. Johnston, and K. C. Tan, "Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming," *IEEE Trans. Evol. Comput.*, vol. 18, no. 2, pp. 193–208, 2013.
[59] F. Zhang, S. Nguyen, Y. Mei, and M. Zhang, "Learning heuristics for multi-objective dynamic production scheduling problems," in *Genetic Programming for Production Scheduling*. Springer, 2021, pp. 187–210.
[60] O. Nicolotti, V. J. Gillet, P. J. Fleming, and D. V. Green, "Multiobjective optimization in quantitative structure- activity relationships: deriving accurate and interpretable qsars," *Journal of Medicinal Chemistry*, vol. 45, no. 23, pp. 5069–5080, 2002.
[61] C. Hii, D. P. Searson, and M. J. Willis, "Evolving toxicity models using multigene symbolic regression and multiple objectives," *International Journal of Machine Learning and Computing*, vol. 1, no. 1, p. 30, 2011.
[62] A. Tahmassebi and A. H. Gandomi, "Building energy consumption forecast using multi-objective genetic programming," *Measurement*, vol. 118, pp. 164–171, 2018.
[63] F. Zhang, Y. Mei, S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted evolutionary multitask genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, 2021.
[64] S. Luke and L. Panait, "A comparison of bloat control methods for genetic programming," *Evolutionary Computation*, vol. 14, no. 3, pp. 309–344, 2006.
[65] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative co-evolution for uncertain capacitated arc routing problem," *Evolutionary Computation*, 2019.
[66] K. Tang, J. Wang, X. Li, and X. Yao, "A scalable approach to capacitated arc routing problems based on hierarchical decomposition," *IEEE Trans. Cybern.*, vol. 47, no. 11, pp. 3928–3940, 2016.
[67] E. K. Burke, M. Hyde, G. Kendall, and J. Woodward, "A genetic programming hyper-heuristic approach for evolving 2-d strip packing heuristics," *IEEE Trans. Evol. Comput.*, vol. 14, no. 6, pp. 942–958, 2010.
[68] E. K. Burke, M. R. Hyde, G. Kendall, and J. Woodward, "Automating the packing heuristic design process with genetic programming," *Evolutionary computation*, vol. 20, no. 1, pp. 63–89, 2012.
[69] S. Luke, L. Panait, G. Balan, S. Paus, Z. Skolicki, J. Bassett, R. Hubley, and A. Chircop, "Ecj: A java-based evolutionary computation research system," *Downloadable versions and documentation can be found at the following url: http://cs. gmu. edu/eclab/projects/ecj*, 2006.



Shaolin Wang is a third-year PhD student at the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. He holds a Master of Computer Science (2019) from Victoria University of Wellington, a Graduate Diploma in Computer Science (2016) from University of Canterbury, Postgraduate Diploma in Geographic Information System (2014) from University of Otago and a BSc in Geographic Information System (2012) from Nanning Normal University. His research interests are Artificial Intelligence, Evolutionary Computation, Hyper-Heuristic, Routing and Real-World Optimisation.



Yi Mei (M'09-SM'18) received the BSc and PhD degrees from the University of Science and Technology of China, Hefei, China, in 2005 and 2010, respectively. He is currently a Senior Lecturer at the School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand. His research interests include evolutionary scheduling and combinatorial optimisation, machine learning, genetic programming, and hyper-heuristics. He has over 130 fully referred publications, including the top journals in EC and Operations Research such as IEEE TEVC, IEEE TCYB, Evolutionary Computation Journal, European Journal of Operational Research, ACM Transactions on Mathematical Software. He serves as a Vice-Chair of the IEEE CIS Emergent Technologies Technical Committee, and a member of Intelligent Systems Applications Technical Committee. He is an Editorial Board Member/Associate Editor of four International Journals, and a guest editor of a special issue of the Genetic Programming Evolvable Machine journal. He serves as a reviewer of over 30 international journals.



Mengjie Zhang (M'04-SM'10-F'19) received the B.E. and M.E. degrees from Artificial Intelligence Research Center, Agricultural University of Hebei, Baoding, China, and the Ph.D. degree in computer science from RMIT University, Melbourne, VIC, Australia, in 1989, 1992, and 2000, respectively. He is currently a Professor of Computer Science, the Head of the Evolutionary Computation Research Group, and the Associate Dean (Research and Innovation) with the Faculty of Engineering, Victoria University of Wellington, Wellington, New Zealand. His current research interests include evolutionary computation, particularly genetic programming, particle swarm optimization, and learning classifier systems with application areas of image analysis, multiobjective optimization, feature selection and reduction, job-shop scheduling, and transfer learning. He has published over 500 research papers in refereed international journals and conferences. He is a Committee Member of the IEEE NZ Central Section. He is a Fellow of the Royal Society of New Zealand and an IEEE Distinguished Lecturer.